

Spring Boot Interview Questions & Answers

1. What is Spring Boot?

framework that makes easy to build Java-based web applications or microservices quickly.

gives **pre-configured setup** so you don't have to write a **lot of boilerplate code or XML configurations**. You just focus on writing business logic.

- It **auto-configures** everything (like database, server, etc.).
- Comes with an **embedded server** (like Tomcat), so no need to deploy WAR files manually.
- Has **Spring Initializr** to create ready-to-run projects.
- Supports **microservices development** easily.
- Uses **convention over configuration**.

Example:

Just add `@SpringBootApplication`, run the main class, and your app is live.

Crack Tip:

Say – "With Spring Boot, I can create a REST API in minutes with minimal setup, which is perfect for fast development and microservices."

2. What are the key features of Spring Boot?

1. **Auto Configuration**
 - Spring Boot auto-configures beans based on dependencies (no need to write XML/manual config).
2. **Embedded Server**
 - Comes with built-in servers like **Tomcat**, Jetty – just run the app like a Java program.
3. **Starter Dependencies**
 - Predefined dependencies like `spring-boot-starter-web`, so we don't need to manage versions manually.
4. **Spring Initializr**
 - Online tool to generate project structure quickly (with needed dependencies).
5. **Production-Ready Features**
 - Health checks, metrics, logging using **Spring Boot Actuator**.
6. **Minimal Configuration**
 - Uses `application.properties` or `application.yml` for easy settings.
7. **Easy Integration with Spring Ecosystem**
 - Works seamlessly with Spring MVC, Spring Data, Spring Security, etc.

How to say in interview:

"Spring Boot provides all necessary features to develop **stand-alone, production-grade applications** with minimal effort. Its auto-configuration, embedded server, and starter dependencies make development fast and easy."

3. What is Spring Boot Starter?

pre-defined set of dependencies used to quickly set up Spring Boot project for specific functionality. Instead of adding 5–6 separate libraries, you just add **one starter**, and it pulls in everything you need.

Example:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

This brings:

- Spring MVC
 - Jackson (for JSON)
 - Embedded Tomcat
- All with compatible versions.

Common Starters:

- spring-boot-starter-web – for web apps & REST
- spring-boot-starter-data-jpa – for JPA/Hibernate & DB
- spring-boot-starter-security – for security
- spring-boot-starter-test – for unit testing

Interview Tip:

“Starters **save time and avoid version mismatch**. They follow Spring Boot’s goal: *convention over configuration*.”

4. What is Spring Boot Auto Configuration?

Spring Boot automatically configures your application **based on the dependencies present** in your project.

Simple example:

If you add **spring-boot-starter-web**, Spring Boot:

- Configures **Tomcat server**
- Sets up **DispatcherServlet**
- Enables **Spring MVC**

You **don’t** have to write **any extra configuration**. It just works.

How it works internally:

- Uses `@EnableAutoConfiguration` (part of `@SpringBootApplication`)
- It scans all META-INF/spring.factories files to load matching configurations
- Example: If H2 DB is in classpath, it sets up datasource automatically

Interview-ready answer:

“Auto Configuration is Spring Boot’s **way of reducing boilerplate code**. Based on what’s on the classpath, it intelligently sets up beans and config without manual work.”

5. What is the role of `@SpringBootApplication`?

`@SpringBootApplication` is a **convenience annotation** that combines three key Spring annotations:

`@SpringBootApplication` = **`@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`**

1. `@Configuration` – Marks the class as a Spring config class (like XML config). (detailed at end)
2. `@EnableAutoConfiguration` – Enables Spring Boot’s auto-configuration magic.
3. `@ComponentScan` – Tells Spring to scan the package and sub-packages for components (`@Component`, `@Service`, `@Controller`, etc.).



It simplifies setup. Just put it on your main class and run it – Spring Boot takes care of the rest.

“`@SpringBootApplication` is the **entry point** of any Spring Boot app. It brings together configuration, component scanning, and auto-configuration in one annotation for ease of use.”

6. How does Spring Boot handle dependency management?

using a **centralized parent POM** that defines default versions of commonly used libraries.

You don’t have to worry about **version conflicts**. Just add the required starter (like spring-boot-starter-web), and it pulls in all the right versions automatically.

How it works:

- Spring Boot provides a **parent POM**:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.0</version>
</parent>
```

- It includes **dependencyManagement**, which defines versions for Spring, Jackson, Hibernate, etc.
- You don't need to specify versions explicitly for those libraries.

Example without managing versions:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

"Spring Boot handles dependency management using Spring Boot Starter POMs and a parent POM, which **automatically manages library versions and transitive dependencies**. This avoids version conflicts and reduces manual configuration.

7. What are some commonly used Spring Boot starters?

- spring-boot-starter-web – for building web apps and REST APIs (includes Spring MVC, Tomcat)
 - spring-boot-starter-data-jpa – for working with databases using Spring Data JPA and Hibernate
 - spring-boot-starter-security – for adding authentication and authorization
 - spring-boot-starter-test – for unit and integration testing (JUnit, Mockito, etc.)
 - spring-boot-starter-thymeleaf – for server-side HTML rendering
 - spring-boot-starter-actuator – for monitoring and health checks
 - spring-boot-starter-validation – for input validation using Hibernate Validator
-

8. How do you create a Spring Boot project?

Using Spring Initializr (<https://start.spring.io>)

Using Spring Boot CLI

Manually via Maven/Gradle project

9. What is the default embedded server in Spring Boot?

Ans: Tomcat. Others: Jetty, Undertow.

10. Can we change the default port in Spring Boot?

Ans: Yes, via application.properties:

```
server.port=8081
```

11. What is application.properties or application.yml in Spring Boot?

"Both application.properties and application.yml are used **to define Spring Boot configuration settings**

like DB config, server port, logging level etc. The difference lies in format and readability."

- ◇ application.properties
 - Key-value format
server.port=8081
 - Easy and flat structure
 - Good for small configs
- ◇ application.yml
 - YAML format (indentation-based)

```
server:
  port: 8081
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/db
    username: root
```

- Supports hierarchical/nested configs
- More readable for complex settings

"Functionally both are same, and Spring Boot gives priority to application.yml if both are present. You can use either based on preference or project standards."

12. How to enable scheduling in Spring Boot?

Scheduling means running piece of code automatically at fixed time or after a fixed interval — like a timer or alarm.

To enable scheduling in Spring Boot, we use `@EnableScheduling` annotation in the main class and `@Scheduled` on the method we want to run on a schedule.

Steps:

1. Add `@EnableScheduling` in the main class:

```
@SpringBootApplication
@EnableScheduling
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

2. Add `@Scheduled` on the method:

```
@Component
public class MyScheduler {
    @Scheduled(fixedRate = 5000)
    public void runTask() {
        System.out.println("Runs every 5 seconds");
    }
}
```

You can use:

- `fixedRate` – runs at a fixed interval
- `fixedDelay` – waits after method finishes
- `cron` – for cron expressions



This makes task scheduling easy without any external libraries. Just add annotations and it work.

13. How to handle exceptions globally in Spring Boot?

When something goes wrong (like invalid input, null pointer, DB error), you don't want to write try-catch in every controller or service — that's messy. Instead, **catch and handle all errors in one central place**, this is called **global exception handling**. In Spring Boot, we use `@ControllerAdvice` along with `@ExceptionHandler` to handle exceptions globally across all controllers.

```
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleNotFound(ResourceNotFoundException
ex) {
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleGlobal(Exception ex) {
        return new ResponseEntity<>("Something went wrong",
HttpStatus.INTERNAL_SERVER_ERROR);
    }

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleUserNotFound(UserNotFoundException
ex) {
        return new ResponseEntity<>("User not found!", HttpStatus.NOT_FOUND);
    }
}
```

In Controller:

```
@RestController
public class UserController {
    @GetMapping("/users/{id}")
    public User getUser(@PathVariable int id) {
        // This will throw if user not found
        return userService.findById(id);
    }
}
```

In UserService:

```
public User findById(int id) {
    Optional<User> user = userRepo.findById(id);
    return user.orElseThrow(() -> new UserNotFoundException("User not
found"));
}
```

- `@ControllerAdvice` = Tells Spring: This class will handle exceptions for all controllers
- `@ExceptionHandler` = catches specific exceptions
- You return custom messages & status codes
- Clean code: no repeated try-catch, avoids repetition
- Centralized error messages
- Makes your API responses consistent

14. What is Spring Boot Actuator?

Provides built-in endpoints/production-ready features like monitoring, health checks (app health status), and gathering metrics (memory, CPU, JVM stats), beans – loaded Spring beans etc. without writing extra code.

✂ How to enable:

1. Add dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

2. Configure in application.properties:

```
management.endpoints.web.exposure.include=*
```

15. What is the use of @RestController?

used to create REST APIs, Combines @Controller+ @ResponseBody to return JSON/XML directly (return value is automatically converted to JSON or XML)

- Eliminates the need to write @ResponseBody on every method
- Sends data (not HTML) directly to the client

```
@RestController
public class UserController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello User"; // returns plain text or JSON
    }
}
```

16. Difference between @Controller and @RestController?

@Controller:

- Returns HTML views (e.g., Thymeleaf, JSP)
- Used for building **web pages (HTML views, jsp)**
- Common use: MVC web apps
- @ResponseBody Must add manually on each method

@RestController:

- Returns data/response body (JSON/XML)
- Used for building **REST APIs (JSON/XML)**
- **Common Use: Microservices / REST APIs**
- Combines @Controller + @ResponseBody

✓ What is @Configuration?

This class contains beans and settings that should be managed by Spring.

It's basically the **Java-based replacement for old-style applicationContext.xml**.

📦 Before Spring Boot (XML configuration):

```
<beans>
    <bean id="myService" class="com.example.MyService"/>
</beans>
```

This XML file registered beans manually.

💻 Now in Spring Boot using @Configuration:

- @Configuration → tells Spring this class contains beans
- @Bean → tells Spring to create and manage this object

```
@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate (); // Spring will manage this
        as a bean
    }
}
```

Now, you can autowire RestTemplate anywhere Because Spring **registered it as a bean**, thanks to @Configuration.

```
@Autowired
private RestTemplate restTemplate;
```

📧 Without @Configuration, this won't work

If you remove @Configuration, Spring will **not scan** or create the bean from @Bean methods.

```
}  
}
```

"So, we use @Controller for traditional web apps and @RestController for API development where response is data, not views."

Want a quick way to remember it in interviews?

Just say: **"RestController = Controller + @ResponseBody, used for APIs."**

User:

17. How does Spring Boot connect to a database?

ChatGPT:

Here's your **interview-ready and easy-to-remember answer**:

"Spring Boot connects to a database using JDBC or JPA by configuring connection details in application.properties or application.yml, and it auto-configures DataSource and related beans."

Steps to connect:

1. **Add dependency** in pom.xml:

```
xml  
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-data-jpa</artifactId>  
</dependency>  
  
<dependency>  
  <groupId>mysql</groupId>  
  <artifactId>mysql-connector-java</artifactId>  
</dependency>
```

2. **Add DB properties** in application.properties:

```
properties  
spring.datasource.url=jdbc:mysql://localhost:3306/mydb  
spring.datasource.username=root  
spring.datasource.password=yourPassword  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true
```

3. **Create Entity & Repository:**

```
java  
@Entity  
public class User {  
  @Id  
  private Long id;  
  private String name;  
}  
  
public interface UserRepository extends JpaRepository<User, Long> {}
```

"Spring Boot auto-configures the DataSource, EntityManager, and Hibernate when it detects JPA on the classpath and DB config in properties file."

🔥 You don't need to write boilerplate config — that's the power of Spring Boot!

Want to go over how Spring handles transactions or JPA repo methods next?

User:

18. What is Spring Data JPA?

ChatGPT:

Here's a **simple and interview-ready answer**:

"Spring Data JPA is a Spring module that simplifies working with databases using JPA. It removes boilerplate code by providing ready-made repository interfaces to perform CRUD operations."

Key Features:

- Reduces need to write SQL or DAO code
- Built on top of **JPA (Java Persistence API)**
- Works with **Hibernate** by default
- Provides **JpaRepository**, which gives all basic DB operations out of the box

Example:

```
java
@Entity
public class User {
    @Id
    private Long id;
    private String name;
}
```

```
java
public interface UserRepository extends JpaRepository<User, Long> {
    // No need to write any code, methods like save(), findAll(), findById() are ready to use
}
```

You can also create custom queries:

```
java
List<User> findByName(String name);
```

"Spring Data JPA speeds up development by handling all boilerplate CRUD operations. We just define interfaces and method names – Spring generates queries automatically."

Let me know if you want a deep-dive on custom queries or native SQL in Spring Data JPA!

User:

19. How to create custom queries in Spring Data JPA? also custom query means native sql ?

ChatGPT:

Great question! Here's your **interview-ready explanation**:

What is a custom query in Spring Data JPA?

"Custom queries are user-defined queries when the method name is not enough, or you want more control over the query. They can be written using JPQL or native SQL."

1. Custom query using JPQL (object-based, not table-based)

```
java
@Query("SELECT u FROM User u WHERE u.name = :name")
List<User> getUsersByName(@Param("name") String name);
```

- JPQL is based on **entity class & field names**, not table/column names.
- Works with JPA providers like Hibernate.

2. Custom query using native SQL

```
java
```

```
@Query(value = "SELECT * FROM users WHERE name = :name", nativeQuery = true)
List<User> getUsersNative(@Param("name") String name);
```

- nativeQuery = true tells Spring to use raw SQL
- Useful when:
 - You need complex joins
 - You want to use DB-specific functions
 - Performance tuning

✓ Extra Tip – Derived Queries (no query writing at all)

```
java
List<User> findByName(String name);
```

Spring will auto-generate the SQL based on method name.

✦ Interview one-liner:

"Custom queries in Spring Data JPA can be written using JPQL or native SQL using the @Query annotation. Native SQL is used when we need DB-specific or complex queries."

Let me know if you want to add pagination or dynamic query next!

User:

20. What is the role of @Entity, @Id, and @GeneratedValue?

ChatGPT:

Perfect! Here's your **interview-ready, simple but strong answer**:

"These are JPA annotations used to map Java classes to database tables and define primary keys."

◆ @Entity

"Marks the class as a JPA entity, meaning it will be mapped to a table in the database."

```
java
@Entity
public class User {
    ...
}
```

◆ @Id

"Marks a field as the primary key of the entity."

```
java
@Id
private Long id;
```

◆ @GeneratedValue

"Used with @Id to auto-generate primary key values, like auto-increment in DB."

```
java
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Common strategies:

- AUTO – Let JPA choose
- IDENTITY – Use DB's auto-increment
- SEQUENCE – Use DB sequence (mostly in Oracle)
- TABLE – Use a table to store sequence (rare)

✳️ Final one-liner to impress:

"@Entity maps the class to a table, @Id defines the primary key, and @GeneratedValue auto-generates the ID values – all part of JPA for ORM mapping."

Let me know if you want a real example of saving or fetching using these!

User:

21. What is CommandLineRunner in Spring Boot?

ChatGPT:

Here's your **interview-perfect answer** for:

🔍 What is CommandLineRunner in Spring Boot?

"CommandLineRunner is a functional interface in Spring Boot used to run specific code after the application startup — like initial setup, DB seeding, logging, etc."

✅ Key Points:

- Runs **only once** after the application context is loaded
- Used for **startup logic**
- Implement its `run(String... args)` method

🔧 Example:

```
java
@Component
public class MyStartupRunner implements CommandLineRunner {

    @Override
    public void run(String... args) throws Exception {
        System.out.println("Application started. Run this logic...");
        // e.g., initialize DB, call API, etc.
    }
}
```

You can also define it as a bean using lambda:

```
java
@Bean
public CommandLineRunner demo() {
    return args -> {
        System.out.println("Startup logic here...");
    };
}
```

✳️ One-liner to impress:

"CommandLineRunner runs custom logic right after Spring Boot app starts — useful for things like DB setup, cleanup, or boot-time checks."

Let me know if you want to compare it with ApplicationRunner too!

User:

22. What is ApplicationRunner?

ChatGPT:

Here's your **interview-ready and clear answer**:

? **What is ApplicationRunner in Spring Boot?**

"ApplicationRunner is similar to CommandLineRunner, used to execute code after the Spring Boot application starts. The difference is: it gives access to ApplicationArguments, which makes it easier to handle input arguments."

✓ **Key Points:**

- Runs **after the application context is loaded**
- Preferred if you want to access and process **command-line arguments**
- Has a method:
run(ApplicationArguments args)

🔧 **Example:**

```
java
@Component
public class MyAppRunner implements ApplicationRunner {

    @Override
    public void run(ApplicationArguments args) throws Exception {
        System.out.println("Application started with args: " + args.getOptionNames());
    }
}
```

vs **Difference:**

Feature	CommandLineRunner	ApplicationRunner
Method	run(String... args)	run(ApplicationArguments args)
Input Handling	Raw string args	Parsed args (options, non-options)
Use Case	Simple startup logic	When you need to read named arguments

✦ **One-liner to impress:**

"ApplicationRunner is like CommandLineRunner but provides structured access to command-line arguments, useful in real apps where input parsing is needed."

Want a combined use-case or when to prefer which one? Let me know!

User:

23. Difference between CommandLineRunner and ApplicationRunner? above chart is sufficient ?

ChatGPT:

Yes — the **above chart + one-liner explanation** is **100% sufficient for interviews**, especially for 3+ years of experience.

But if you want to **nail it perfectly**, here's a refined and final version you can speak confidently:

✓ **Interview-ready table (short & strong)**

Feature	CommandLineRunner	ApplicationRunner
Method	run(String... args)	run(ApplicationArguments args)
Argument Type	Raw String array	Parsed arguments (options, flags, etc.)

Feature	CommandLineRunner	ApplicationRunner
When to Use	Simple logic, no argument parsing	Need structured access to input args
Input Handling	Manual parsing	Built-in helper methods like <code>getOptionNames()</code>

🚀 One-liner to end strong:

"Both run after app starts, but ApplicationRunner gives parsed arguments — useful when input needs to be read in a structured way."

✅ This is precise, practical, and impresses with real-use awareness.

Let's go for the next one!

User:

23. What is `@SpringBootTest`?

ChatGPT:

Here's your **interview-ready answer** for:

? *What is @SpringBootTest in Spring Boot?*

"@SpringBootTest is a testing annotation that loads the full Spring ApplicationContext, allowing you to test your application's integration with all its components."

✅ Key Points:

- Used for **integration testing**, not just unit testing
- Boots up the entire Spring Boot application like it runs in real
- Allows testing of:
 - REST endpoints
 - Services
 - Repositories
 - Security, filters, configs, etc.

🔧 Example:

```
java
@SpringBootTest
public class UserServiceTest {

    @Autowired
    private UserService userService;

    @Test
    void testGetUser() {
        User user = userService.getUser(1L);
        assertEquals("John", user.getName());
    }
}
```

💡 Behind the scenes:

- Loads full Spring container
- Automatically mocks embedded server (like Tomcat) if needed

🚀 Final one-liner:

"We use `@SpringBootTest` when we want to test the entire flow of the application, not just isolated units — it's great for real-world scenario testing."

Let me know if you want to add mock examples or compare it with `@WebMvcTest` next!

User:

24. What is `@DataJpaTest`?

ChatGPT:

Here's your **interview-ready and beginner-friendly answer** for:

? *What is `@DataJpaTest` in Spring Boot?*

"`@DataJpaTest` is used for testing JPA components like repositories. It loads only the JPA-related part of the Spring context — not the full app."

✅ Key Features:

- Tests only **Repository layer** (like `UserRepository`)
- Uses **in-memory DB** (like H2) by default
- Automatically **rolls back** transactions after each test
- Fast and lightweight compared to `@SpringBootTest`

🔧 Example:

```
java
@DataJpaTest
public class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

    @Test
    void testSaveUser() {
        User user = new User("John", "john@example.com");
        userRepository.save(user);

        assertEquals(1, userRepository.findAll().size());
    }
}
```

🆚 Compared to `@SpringBootTest`:

Feature	@SpringBootTest	@DataJpaTest
Loads full context	✅ Yes	❌ Only JPA components
Test type	Full integration test	Repository/unit test
Speed	Slower	Faster
DB	Real or in-memory	In-memory (default H2)

🌟 One-liner to end:

"`@DataJpaTest` is perfect for testing JPA repositories in isolation with in-memory DB, making tests faster and cleaner."

Want to go over `@WebMvcTest` next or mock testing with `@MockBean`?

User:

25. How to run Spring Boot app as a JAR?

ChatGPT:

Here's your **interview-ready, to-the-point answer**:

? **How to run a Spring Boot app as a JAR?**

"We can run a Spring Boot app as a JAR by building it with Maven/Gradle, and then using the `java -jar` command."

✓ **Steps:**

1. **Build the JAR** using Maven:

```
bash
mvn clean install
```

This creates a JAR file in the `target/` directory.

2. **Run the JAR:**

```
bash
java -jar target/myapp-0.0.1-SNAPSHOT.jar
```

🔧 **Behind the scenes:**

- Spring Boot uses an embedded server (like Tomcat)
 - No need to deploy WAR on external server
 - The JAR contains everything: app code + server + dependencies
-

✳️ **One-liner for interview:**

"Spring Boot creates an executable JAR with embedded server, so we just run it using `java -jar` — no external deployment needed."

Want a quick Gradle version or Docker version too?

User:

26. How to disable a specific auto-configuration?

ChatGPT:

Here's your **interview-ready answer** for:

? **How to disable a specific auto-configuration in Spring Boot?**

"We can disable any specific auto-configuration class using the `exclude` attribute of `@SpringBootApplication` or with `spring.autoconfigure.exclude` in properties."

✓ **Option 1: Using @SpringBootApplication**

```
java
@SpringBootApplication(exclude = { DataSourceAutoConfiguration.class })
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

- Useful when you don't want Spring to auto-configure a `DataSource`, `Security`, etc.
-

✓ **Option 2: Using application.properties**

```
properties
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

Common use cases:

- You don't want a DataSource auto-configured (using only JPA with external config)
- You want full control over Security, so disable SecurityAutoConfiguration

One-liner to impress:

"To disable auto-config, use exclude in @SpringBootApplication or properties — this gives us manual control over specific components."

Want to see how to conditionally enable beans using @ConditionalOnMissingBean too?

User:

27. What is the use of profiles in Spring Boot?

ChatGPT:

Here's your **interview-ready answer** for:

What is the use of Profiles in Spring Boot?

"Spring Boot Profiles allow us to define different configurations for different environments — like dev, test, and prod — and activate them as needed."

Why use profiles?

- To separate environment-specific settings
- Avoid hardcoding database URLs, ports, log levels, etc.

How to create profiles:

1. Create config files like:
 - application-dev.properties
 - application-prod.properties
2. In each file, define environment-specific settings:

```
properties
# application-dev.properties
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/devdb
```

```
properties
# application-prod.properties
server.port=9090
spring.datasource.url=jdbc:mysql://prod-server/db
```

How to activate a profile:

- In application.properties:

```
properties
spring.profiles.active=dev
```

- Or via command line:

```
bash
```

```
java -jar app.jar --spring.profiles.active=prod
```

✦ One-liner to impress:

"Spring Profiles help manage different configs for different environments — we just activate the needed one using `spring.profiles.active`."

Let me know if you want real example files or conditional beans using profiles!

User:

28. How to set active profile?

ChatGPT:

Here's your **interview-ready and simple answer** for:

? *How to set active profile in Spring Boot?*

"We can set the active profile using `spring.profiles.active` in properties, command line, or environment variable — this tells Spring Boot which config to load."

✓ 1. In `application.properties` (static way):

```
properties
spring.profiles.active=dev
```

Loads `application-dev.properties` or `application-dev.yml`

✓ 2. Using command line (runtime switch):

```
bash
java -jar myapp.jar --spring.profiles.active=prod
```

Useful when deploying to different environments (CI/CD, Docker, etc.)

✓ 3. Using environment variable:

```
bash
export SPRING_PROFILES_ACTIVE=qa
```

Especially helpful in cloud and container deployments

✦ One-liner to impress:

"We set the active profile using `spring.profiles.active`, which controls which environment-specific configuration Spring Boot uses."

Want to go next with how to load profile-specific beans or config classes?

User:

29. What is the default scope of Spring Beans in Spring Boot?

ChatGPT:

Here's your **interview-cracking, clear answer** for:

? *What is the default scope of Spring Beans in Spring Boot?*

"The default scope of a Spring Bean is singleton, which means a single instance is created per Spring container and shared across the application."

✓ Explanation:

- When you annotate a class with `@Component`, `@Service`, `@Repository`, etc., Spring automatically registers it as a **singleton bean** (by default).
 - Only **one object** is created and reused wherever it's injected using `@Autowired`.
-

🔧 Example:

```
java
@Component
public class MyBean {
    // Only one instance will be created
}
```

```
java
@Autowired
private MyBean myBean; // same instance everywhere
```

📦 Other scopes (optional):

- `prototype` – New instance every time it's requested
- `request` – One per HTTP request (Web only)
- `session` – One per HTTP session
- `application` – One per ServletContext
- `websocket` – One per WebSocket session

You set them using:

```
java
@Scope("prototype")
```

✳️ One-liner to impress:

"Spring beans are singleton by default, meaning one instance is shared across the app unless a different scope is explicitly defined."

Want to dive into prototype scope behavior with `@Autowired` next?

User:

30. What is the use of `@Value` annotation?

ChatGPT:

Here's your **interview-ready, clear and short answer** for:

? What is the use of `@Value` annotation in Spring Boot?

"`@Value` is used to inject values from property files or environment variables into Spring beans."

✓ Common Use Cases:

- Inject values from `application.properties` or `application.yml`
 - Inject default values
 - Inject system/environment variables
-

🔧 Example 1: Inject from application.properties

```
properties
app.name=MySpringApp
```

```
java
@Value("${app.name}")
private String appName;
```

🔧 Example 2: Inject with default value

```
java
@Value("${app.version:1.0}")
private String version;
```

If app.version is not found, 1.0 will be used.

🔧 Example 3: Inject system property or env variable

```
java
@Value("${JAVA_HOME}")
private String javaHome;
```

📌 One-liner to impress:

"@Value lets us pull values from config files or environment directly into variables, making the code configurable and environment-aware."

Let me know if you want @ConfigurationProperties vs @Value next!

User:

31. What is @ConfigurationProperties?

ChatGPT:

Here's your **interview-ready and simplified answer** for:

? What is @ConfigurationProperties in Spring Boot?

"@ConfigurationProperties is used to bind multiple related properties from application.properties or application.yml into a POJO class."

✅ Why use it?

- Cleanly maps a **group of properties** into a class
- Better for **structured configs** than using multiple @Value
- Helps in keeping configuration **type-safe and organized**

🔧 Example:

1. application.yml:

```
yaml
app:
  name: MyApp
  version: 1.0
  author: pratiksha
```

2. Create POJO:

```

java
@Component
@ConfigurationProperties(prefix = "app")
public class AppConfig {
    private String name;
    private String version;
    private String author;

    // getters and setters
}

```

Spring will automatically bind all `app.*` properties to this class.

✓ Don't forget:

- Add `@EnableConfigurationProperties` in config class (if not using `@Component`)
- Or annotate class with `@Component` directly

✦ One-liner to impress:

"@ConfigurationProperties helps map grouped config values into a bean for cleaner, maintainable, and type-safe configuration management."

Want to compare `@Value` vs `@ConfigurationProperties` next?

User:

yes

ChatGPT:

Here's your **interview-perfect comparison of @Value vs @ConfigurationProperties**:

✓ @Value vs @ConfigurationProperties

Feature	@Value	@ConfigurationProperties
Purpose	Inject a single property	Bind a group of related properties
Use case	Quick, one-off values	Structured config (nested/grouped)
Type-safety	Limited (String unless casted manually)	Full type-safe binding (with lists, maps, objects)
Ideal for	Simple or constant values	Complex configs like DB, server, third-party settings
Supports nested properties	✗ No	✓ Yes
Default value support	✓ Yes (<code>@Value("\${key:default}")</code>)	✗ Not directly, but can be handled in class logic
Validation support	✗ No	✓ Yes (with <code>@Validated</code> and JSR-303 annotations)
Boilerplate	Less	More (requires POJO + getters/setters)

🔧 Example:

@Value:

```

properties
app.name=MyApp

```

```

java
@Value("${app.name}")
private String appName;

```

@ConfigurationProperties:

```

properties
app.name=MyApp
app.version=1.0

```



```
java
@Component
@ConfigurationProperties(prefix = "app")
public class AppConfig {
    private String name;
    private String version;
    // Getters & Setters
}
```

✦ One-liner to impress:

"@Value is good for single properties, while @ConfigurationProperties is ideal for grouped, type-safe, and maintainable configurations."

Would you like a real-world use case for @ConfigurationProperties, like DB or S3 configs?

User:

32. What is @EnableConfigurationProperties?

ChatGPT:

Here's your **interview-cracking and easy answer** for:

? What is @EnableConfigurationProperties in Spring Boot?

"@EnableConfigurationProperties is used to enable binding of external configuration properties to a POJO using @ConfigurationProperties."

✓ When is it needed?

- **Only needed** when your @ConfigurationProperties class is **not annotated with @Component**
- It tells Spring: **"Hey, treat this class as a configuration bean"**

🔧 Example:

1. AppConfig.java (NO @Component)

```
java
@ConfigurationProperties(prefix = "app")
public class AppConfig {
    private String name;
    private String version;
    // getters & setters
}
```

2. Register it in a config class:

```
java
@Configuration
@EnableConfigurationProperties(AppConfig.class)
public class MyConfig {
    // other configs
}
```

✓ Now Spring will load and bind app.* properties into AppConfig.

💡 Pro Tip:

If you use @Component on the class, **you don't need @EnableConfigurationProperties**.

✦ One-liner to impress:

"@EnableConfigurationProperties registers a POJO with @ConfigurationProperties so Spring can bind external properties — it's needed only if no @Component is used."

Need to see how it works with validation or custom property files?

User:

34. What is the purpose of @EnableAutoConfiguration?

ChatGPT:

Here's your **interview-ready and beginner-friendly answer** for:

? What is the purpose of @EnableAutoConfiguration in Spring Boot?

"@EnableAutoConfiguration tells Spring Boot to automatically configure beans based on the classpath, defined beans, and configuration properties."

✓ Why is it important?

- It **reduces boilerplate** — no need to define every bean manually
- Spring Boot **intelligently configures** components (like DataSource, Tomcat, Jackson, etc.) if it finds them in the classpath

🔧 Example:

```
java
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

Internally, @SpringBootApplication includes:

```
java
@EnableAutoConfiguration
@ComponentScan
@Configuration
```

So, this:

```
java
@EnableAutoConfiguration
```

- Will auto-configure things like:
 - Embedded Tomcat
 - Spring MVC setup
 - Jackson for JSON
 - H2 or MySQL DataSource (if present in classpath)

✓ Common with exclusions:

```
java
@EnableAutoConfiguration(exclude = { DataSourceAutoConfiguration.class })
```

🌟 One-liner to impress:

"@EnableAutoConfiguration is the core of Spring Boot's magic — it auto-configures beans based on classpath and config so you don't have to."

Want to go over how it works internally using spring.factories or conditions like @ConditionalOnClass?

User:

35. What is the order of precedence for Spring Boot configurations?

ChatGPT:

Here's your **interview-ready answer** for:

? What is the order of precedence for Spring Boot configurations?

"Spring Boot loads configuration properties in a specific order — values defined later override earlier ones."

✓ Order of Precedence (High → Low):

1. **Command line arguments**
--server.port=9090
Highest priority
 2. **application.properties or application.yml** (inside src/main/resources)
 3. **Profile-specific files**
e.g., application-dev.properties
 4. **Environment variables**
export SERVER_PORT=9090
 5. **JVM system properties**
-Dserver.port=9090
 6. **@PropertySource annotations** on @Configuration classes
 7. **Default properties** set in SpringApplication.setDefaultProperties()
-

🔧 Example:

If server.port is defined:

- In application.properties as 8080
- And also passed via command line as --server.port=9090

👉 Spring Boot uses **9090**, because **command line overrides everything**.

✳ One-liner to impress:

"Spring Boot follows a well-defined config loading order, and the last source to define a property wins — command-line args have the highest precedence."

Need a tip on how to override properties per environment or in tests?

User:

36. What is a bean?

ChatGPT:

Here's your **simple and interview-perfect answer** for:

? What is a bean in Spring Boot?

"A bean is an object that is managed by the Spring container — it's created, configured, and injected automatically when needed."

✓ Why are beans important?

- Spring manages the **lifecycle**, **dependencies**, and **scope** of beans

- Helps in building **loosely coupled** and **testable** applications

✅ How do you define a bean?

1. Using annotations:

```
java
@Component
public class MyService {
    // Automatically managed as a bean
}
```

Also via:

- @Service, @Repository, @Controller (specialized @Component)

2. Using @Bean method in config class:

```
java
@Configuration
public class AppConfig {
    @Bean
    public MyBean myBean() {
        return new MyBean();
    }
}
```

📦 Spring manages:

- **Creation** of the object
- **Dependency injection**
- **Scope** (singleton/prototype/request/etc.)
- **Destruction** (if needed)

🚀 One-liner to impress:

"A bean is a Spring-managed object — you just define it and Spring takes care of wiring, lifecycle, and scope."

Want to dive into types of bean scopes next?

User:

37. What is the difference between @Component, @Service, @Repository, and @Controller?

ChatGPT:

Here's your **interview-ready and beginner-friendly answer** for:

? What is the difference between @Component, @Service, @Repository, and @Controller in Spring Boot?

All are Spring stereotypes that make a class a Spring bean — but each has a specific role in layered architecture.

✅ Summary Table:

Annotation	Layer / Purpose	Notes
@Component	Generic component (default)	Base annotation; used when no specific role
@Service	Business logic layer	Indicates service class; same as @Component + semantic clarity
@Repository	Data Access Layer (DAO)	Converts DB exceptions into Spring DataAccessException (extra benefit)
@Controller	Web Layer (MVC Controller)	Used for handling HTTP requests and returning views

✅ Use Cases:

@Component – Generic Spring-managed bean:

```
java
@Component
public class EmailValidator { }
```

@Service – Business logic or service class:

```
java
@Service
public class UserService { }
```

@Repository – Database interaction class:

```
java
@Repository
public class UserRepository {
    // Handles persistence logic
}
```

Also provides automatic exception translation

@Controller – Handles HTTP requests:

```
java
@Controller
public class HomeController {
    @GetMapping("/")
    public String home() {
        return "index"; // returns view
    }
}
```

Use @RestController if returning JSON

✦ One-liner to impress:

"@Component is the generic stereotype, while @Service, @Repository, and @Controller are specialized for clarity and layer-specific behavior."

Want to go next with @RestController vs @Controller or how Spring creates beans behind the scenes?

User:

38. How to consume REST API using RestTemplate?

ChatGPT:

Here's your **interview-ready answer with code** for:

🔗 **How to consume REST API using RestTemplate in Spring Boot?**

"RestTemplate is a Spring class used to make HTTP requests to external REST APIs — like GET, POST, PUT, DELETE."

✅ **Steps to Use RestTemplate:**

1. Create a RestTemplate bean

```
java
@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

2. Autowire and use it:

```
java
@Service
public class UserService {

    @Autowired
    private RestTemplate restTemplate;

    public User getUserById(int id) {
        String url = "https://api.example.com/users/" + id;
        return restTemplate.getForObject(url, User.class);
    }
}
```

✓ Common Methods:

Method	Purpose
<code>getForObject(url, class)</code>	GET request — returns object directly
<code>getForEntity(url, class)</code>	GET — returns full response (body + status)
<code>postForObject(url, req, class)</code>	POST request, gets response object
<code>exchange(...)</code>	For advanced cases (headers, HTTP method, etc.)

🔧 Example: POST API Call

```
java
User newUser = new User("Alice", "alice@example.com");

User created = restTemplate.postForObject(
    "https://api.example.com/users",
    newUser,
    User.class
);
```

✳ One-liner to impress:

"RestTemplate is a simple and powerful way to call REST APIs in Spring — just autowire it and use methods like `getForObject()` or `postForObject()`."

Want me to show how to use `exchange()` with headers or switch to `WebClient` (newer alternative)?

User:

39. What is `WebClient` in Spring Boot?

ChatGPT:

Here's your **interview-ready and beginner-friendly answer** for:

? What is `WebClient` in Spring Boot?

"`WebClient` is the modern, non-blocking, reactive alternative to `RestTemplate` for calling REST APIs in Spring."

✓ Why use `WebClient`?

- **Asynchronous & Non-blocking**
- Suitable for **reactive programming**
- Replaces `RestTemplate` (which is now in maintenance mode)
- Can handle large volumes of concurrent HTTP calls efficiently

🔧 Basic Usage Example:

1. Add Bean (Optional)

```
java
@Bean
public WebClient webClient(WebClient.Builder builder) {
    return builder.baseUrl("https://api.example.com").build();
}
```

2. Use WebClient

```
java
@Service
public class UserService {

    @Autowired
    private WebClient webClient;

    public Mono<User> getUserById(String id) {
        return webClient.get()
            .uri("/users/{id}", id)
            .retrieve()
            .bodyToMono(User.class);
    }
}
```

`Mono<User>` means it will return a single User object **asynchronously**

✅ Key Methods:

Method	Description
<code>.get(), .post()</code>	HTTP method
<code>.uri(...)</code>	Set the endpoint
<code>.retrieve()</code>	Trigger the call
<code>.bodyToMono()</code>	Convert response to object
<code>.bodyToFlux()</code>	For a list of responses

✅ When to prefer WebClient over RestTemplate?

Feature	RestTemplate	WebClient (✅ Modern)
Blocking	✅ Yes	❌ No (Non-blocking)
Reactive support	❌ No	✅ Yes
Performance	Limited for many calls	✅ Efficient concurrency

🔥 One-liner to impress:

"WebClient is Spring's reactive HTTP client — ideal for non-blocking, high-performance REST calls in modern applications."

Need help converting a RestTemplate call to WebClient?

User:

40. What is Spring WebFlux?

ChatGPT:

Here's your **interview-focused and beginner-friendly answer** for:

? What is Spring WebFlux?

"Spring WebFlux is Spring's reactive, non-blocking web framework — built for handling asynchronous, high-concurrency applications using reactive programming."

✅ Key Highlights:

- Introduced in **Spring 5**
- Built on **Project Reactor** (uses Mono, Flux)
- Ideal for **event-driven, streaming, or high-load applications**
- Works with **WebClient** (instead of RestTemplate)
- Supports **annotation-based** and **functional** programming models

✅ When to use WebFlux?

Use Case	WebFlux Advantage
1000s of concurrent users	✅ Non-blocking, lightweight
Streaming or real-time data (like chat, stock updates)	✅ Reactive streams support
Microservices communication	✅ Uses WebClient for async calls
Reactive NoSQL or reactive DBs (like MongoDB Reactive)	✅ Fully reactive chain

🔧 Basic Controller Example:

```
java
@RestController
public class UserController {

    @GetMapping("/user")
    public Mono<User> getUser() {
        return Mono.just(new User("Pratiksha", "pune@example.com"));
    }

    @GetMapping("/all-users")
    public Flux<User> getAllUsers() {
        return Flux.just(
            new User("A", "a@mail.com"),
            new User("B", "b@mail.com")
        );
    }
}
```

- Mono<T> → emits **0 or 1 item**
- Flux<T> → emits **0 to many items**

✅ Difference: Spring MVC vs WebFlux

Feature	Spring MVC	Spring WebFlux
Model	Servlet (blocking)	Reactive (non-blocking)
Concurrency	Thread per request	Event-loop, efficient
Return types	Objects / ResponseEntity	Mono, Flux
HTTP Client	RestTemplate	WebClient

🚀 One-liner to impress:

"Spring WebFlux is a fully non-blocking web framework built for reactive applications using Mono and Flux to handle massive concurrency efficiently."

Would you like to know when not to use WebFlux or real project examples using it?

User:

41. What is @Bean annotation?

ChatGPT:

Here's your **interview-perfect and beginner-friendly answer** for:

? What is @Bean annotation in Spring Boot?

"@Bean is used to explicitly declare a Spring bean inside a @Configuration class — Spring will manage its lifecycle and dependencies."

✓ When to use @Bean?

- When you want to **register a third-party class** or **custom object** as a Spring bean but **can't use annotations like @Component**
- Ideal for **external libraries** or objects created with special constructor logic

🔧 Example:

```
java
@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

Now, RestTemplate can be auto-wired in other classes.

```
java
@Autowired
private RestTemplate restTemplate;
```

✓ Key Points:

- Defined inside a class annotated with @Configuration
- Return type = bean object
- Spring creates and manages this object **as a singleton by default**

✓ Difference from @Component?

Feature	@Component	@Bean
Where used	On class	On method inside @Configuration
Target class	Your own class	Often used for 3rd-party classes
Instantiation	Automatic	Manual via method

🌟 One-liner to impress:

"@Bean lets you manually define a Spring-managed object in a @Configuration class — useful for objects you can't annotate directly."

Want to dive into bean scopes or how to inject dependencies in @Bean methods?

User:

42. What is DevTools in Spring Boot?

ChatGPT:

Here's your **interview-ready and simple answer** for:

? What is Spring Boot DevTools?

"DevTools is a Spring Boot module that improves developer productivity by enabling automatic restarts, live reload, and fast feedback during development."

✓ Key Features of DevTools:

Feature	Description
 Automatic Restart	App restarts automatically when you change Java files or classpath resources
 LiveReload Support	Reloads browser automatically if HTML/CSS changes (if LiveReload plugin installed)
 Caching Disabled	Disables template caching (Thymeleaf, Freemarker) for faster feedback
 H2 Console Enabled	Automatically enables H2 DB console (if used)
 Dev vs Prod Settings	Only active in dev — never affects production builds

✓ How to Enable:

Just add this in `pom.xml` (for Maven):

```
xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
</dependency>
```

No extra config needed — it activates automatically in **development mode**.

✗ Not active in production:

- DevTools is **disabled in production** JARs by default.
- Spring detects if the application is running from a **fully packaged JAR** and skips DevTools.

✦ One-liner to impress:

"DevTools boosts development speed by enabling auto-restart, live reload, and disabling caches — but it's automatically turned off in production."

Want a tip on how to safely reload without full restart or how to combine it with IntelliJ?

User:

43. How to handle cross-origin requests (CORS)?

ChatGPT:

Here's your **interview-perfect and practical answer** for:

? How to handle cross-origin requests (CORS) in Spring Boot?

"CORS (Cross-Origin Resource Sharing) allows a frontend from one domain (like `localhost:4200`) to access backend APIs from another domain (like `localhost:8080`). In Spring Boot, you can handle it globally or at the controller level using `@CrossOrigin` or `WebMvcConfigurer`."

✓ 1. Using `@CrossOrigin` Annotation (Quick & Easy)

a. On method level:

```
java
@CrossOrigin(origins = "http://localhost:4200")
@GetMapping("/users")
public List<User> getUsers() {
    return userService.getAllUsers();
}
```

b. On controller class level:

```
java
@CrossOrigin(origins = "*") // Allow all origins
@RestController
public class UserController {
    // All endpoints inside allow CORS
}
```

✓ 2. Global CORS Configuration (Best Practice for real projects)

```
java
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**") // apply to all paths
            .allowedOrigins("http://localhost:4200")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*");
    }
}
```

✦ When is this needed?

- Frontend (Angular/React) runs on a **different port** than Spring Boot backend
- Without CORS config, browser will **block the request** due to security policies

✓ One-liner to impress:

"CORS in Spring Boot can be handled using @CrossOrigin or WebMvcConfigurer, which allows secure cross-domain communication between frontend and backend."

Want a sample use case where you had to enable CORS in your project?

User:

44. How to configure custom error pages?

ChatGPT:

Here's your **interview-ready, beginner-friendly answer** for:

? How to configure custom error pages in Spring Boot?

"In Spring Boot, we can configure custom error pages using error.html, ErrorController, or through specific HTTP status code mappings."

✓ 1. Simple Way: error.html in /resources/templates/

Spring Boot automatically maps errors to this file.

```
html
<!-- resources/templates/error.html (for Thymeleaf) -->
<!DOCTYPE html>
<html>
<head><title>Error</title></head>
<body>
    <h2>Something went wrong!</h2>
    <p>Please try again later.</p>
</body>
</html>
```

This will be shown for **any error code (e.g., 404, 500)** unless overridden.

✅ 2. Custom Error Page per HTTP Status

Create files like:

```
swift
/resources/public/error/404.html
/resources/public/error/500.html
```

Spring Boot will serve 404.html for 404 errors, 500.html for 500 errors, etc.

✅ 3. Using @Controller with ErrorController (Advanced)

For full control over error handling:

```
java
@Controller
public class CustomErrorController implements ErrorController {

    @RequestMapping("/error")
    public String handleError(HttpServletRequest request) {
        Object status = request.getAttribute(RequestDispatcher.ERROR_STATUS_CODE);

        if (status != null) {
            int code = Integer.parseInt(status.toString());
            if (code == 404) return "error-404";
            else if (code == 500) return "error-500";
        }
        return "error";
    }
}
```

You can then create error-404.html, error-500.html in templates/.

🌟 One-liner to impress:

"Custom error pages in Spring Boot can be configured using static HTML files or programmatically using ErrorController for more control over error handling."

Want a sample for handling errors in REST APIs with custom JSON response too?

User:

45. What is Spring Security?

ChatGPT:

Here's your **interview-ready, beginner-friendly answer** for:

? What is Spring Security?

"Spring Security is a powerful and customizable framework that handles authentication, authorization, and protection against common security threats in Spring applications."

✅ Key Features of Spring Security:

Feature	Purpose
🔑 Authentication	Verifies who the user is (login, username/password)
✅ Authorization	Controls what the user can access (roles, permissions)
🛡️ CSRF Protection	Prevents Cross-Site Request Forgery attacks
🔑 Session Management	Manages login sessions, timeouts, etc.
🔒 Password Encoding	Encrypts passwords securely (e.g., BCrypt)
📦 JWT & OAuth Support	Supports token-based auth (JWT), OAuth2, SSO, etc.

✅ Basic Authentication Example:

```

java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests()
                .requestMatchers("/admin").hasRole("ADMIN")
                .requestMatchers("/user").hasRole("USER")
                .anyRequest().authenticated()
            .and()
                .formLogin(); // Enables default login page

        return http.build();
    }
}

```

✓ In application.properties:

```

properties
spring.security.user.name=admin
spring.security.user.password=admin123

```

Spring will automatically apply security to all endpoints and show a login page.

🔵 Common Attacks Spring Security Protects Against:

- CSRF (Cross-Site Request Forgery)
- XSS (Cross Site Scripting)
- Session Fixation
- Clickjacking

🔴 One-liner to impress:

"Spring Security handles login, access control, and common security risks, and integrates easily with JWT, OAuth, and role-based access."

Need help showing how you implemented JWT with Spring Security in your project?

User:

46. How to secure REST endpoints in Spring Boot?

ChatGPT:

Here's your **interview-ready and easy-to-grasp** answer for:

🔴 ? How to secure REST endpoints in Spring Boot?

"We secure REST endpoints using Spring Security by configuring URL access rules, authentication mechanisms (like JWT, Basic Auth), and roles."

✓ 1. Basic Security (Role-Based Access Control)

```

java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests()

```

```

        .requestMatchers("/admin/**").hasRole("ADMIN")
        .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
        .anyRequest().authenticated()
        .and()
        .httpBasic(); // or formLogin()

    return http.build();
}
}

```

- `hasRole("ADMIN")` → only users with `ROLE_ADMIN` can access
- `.authenticated()` → all users must be logged in

✓ 2. Securing with JWT (Token-Based Auth)

- Add a JWT filter to intercept requests and extract user info from token
- No session; completely stateless

```

java
http
.csrf().disable()
.authorizeHttpRequests()
    .requestMatchers("/auth/**").permitAll()
    .anyRequest().authenticated()
    .and()
    .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);

```

Common in Microservices and Angular/React-based apps

✓ 3. Method-Level Security with `@PreAuthorize`

```

java
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/admin/data")
public String getAdminData() {
    return "Sensitive data";
}

```

Enable it with:

```

java
@EnableMethodSecurity // Spring Boot 3+

```

✓ 4. Custom Login & Exception Handling

You can also define:

- Custom login pages
- Access denied pages
- Exception handlers for unauthorized access

✦ One-liner to impress:

"REST endpoints in Spring Boot are secured using Spring Security by defining URL-based or method-level access rules, and commonly enhanced using JWT for stateless APIs."

Want a quick JWT setup walkthrough or code for token validation?

User:

47. How to implement JWT with Spring Boot?

ChatGPT:

Here's your **interview-cracking answer with step-by-step clarity** for:

? **How to implement JWT in Spring Boot?**

"JWT (JSON Web Token) is used to secure REST APIs in a stateless way. In Spring Boot, it's implemented by generating a token on login, then validating that token for every API request."

✔ **Step-by-Step JWT Implementation:**

Step 1: Add Dependencies (Maven)

```
xml
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt</artifactId>
  <version>0.9.1</version>
</dependency>
```

Step 2: Create JWT Utility Class

```
java
@Component
public class JwtUtil {
    private final String SECRET = "mySecretKey";

    public String generateToken(String username) {
        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60)) // 1 hr
            .signWith(SignatureAlgorithm.HS256, SECRET)
            .compact();
    }

    public String extractUsername(String token) {
        return Jwts.parser().setSigningKey(SECRET)
            .parseClaimsJws(token).getBody().getSubject();
    }

    public boolean validateToken(String token, UserDetails userDetails) {
        String username = extractUsername(token);
        return username.equals(userDetails.getUsername());
    }
}
```

Step 3: Create Authentication Endpoint

```
java
@RestController
public class AuthController {

    @Autowired AuthenticationManager authManager;
    @Autowired JwtUtil jwtUtil;
    @Autowired UserDetailsService userDetailsService;

    @PostMapping("/authenticate")
    public ResponseEntity<String> authenticate(@RequestBody AuthRequest request) {
        Authentication auth = authManager.authenticate(
            new UsernamePasswordAuthenticationToken(request.getUsername(), request.getPassword()));

        final UserDetails userDetails = userDetailsService.loadUserByUsername(request.getUsername());
        final String token = jwtUtil.generateToken(userDetails.getUsername());

        return ResponseEntity.ok(token);
    }
}
```

Step 4: Create JWT Filter

```

java
@Component
public class JwtFilter extends OncePerRequestFilter {

    @Autowired JwtUtil jwtUtil;
    @Autowired UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
        throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");
        String token = null;
        String username = null;

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            token = authHeader.substring(7);
            username = jwtUtil.extractUsername(token);
        }

        if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails = userDetailsService.loadUserByUsername(username);
            if (jwtUtil.validateToken(token, userDetails)) {
                UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                    userDetails, null, userDetails.getAuthorities());
                authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }

        chain.doFilter(request, response);
    }
}

```

Step 5: Configure Spring Security

```

java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired JwtFilter jwtFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf().disable()
            .authorizeHttpRequests()
                .requestMatchers("/authenticate").permitAll()
                .anyRequest().authenticated()
            .and()
            .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            .and()
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }

    @Bean
    public AuthenticationManager authManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }
}

```

🌟 One-liner to impress:

"JWT in Spring Boot is implemented by generating a token during login, and validating it using a custom filter for every secured API request — making the app stateless and secure."

Need a full project structure or Postman test flow for this setup?

User:

48. How to enable HTTPS in Spring Boot?

ChatGPT:

Here's your **interview-perfect answer** for:

? **How to enable HTTPS in Spring Boot?**

"To enable HTTPS in Spring Boot, you need to generate a keystore (SSL certificate) and configure HTTPS settings in application.properties or application.yml."

✓ **Step-by-Step to Enable HTTPS:**

♦ **Step 1: Generate Keystore**

Use Java keytool to generate a .jks file:

```
bash
keytool -genkeypair -alias myssl -keyalg RSA -keysize 2048 -storetype JKS -keystore mykeystore.jks -validity 3650
```

🔑 It will ask for password, name, org details etc.

♦ **Step 2: Place the .jks file**

Put mykeystore.jks in src/main/resources (or external path).

♦ **Step 3: Add HTTPS Config in application.properties**

```
properties
server.port=8443
server.ssl.key-store=classpath:mykeystore.jks
server.ssl.key-store-password=yourPassword
server.ssl.key-alias=myssl
```

You can also use application.yml for the same config.

♦ **Step 4 (Optional): Redirect HTTP to HTTPS**

Create a custom WebServerFactory:

```
java
@Bean
public WebServerFactoryCustomizer<TomcatServletWebServerFactory> servletContainer() {
    return server -> {
        Connector connector = new Connector(TomcatServletWebServerFactory.DEFAULT_PROTOCOL);
        connector.setScheme("http");
        connector.setPort(8080);
        connector.setSecure(false);
        connector.setRedirectPort(8443);
        server.addAdditionalTomcatConnectors(connector);
    };
}
```

✓ **Result:**

- HTTPS enabled on https://localhost:8443
- Optional HTTP redirect from http://localhost:8080

✳ **One-liner to impress:**

"Spring Boot enables HTTPS using a self-signed certificate (keystore), configured via properties, and optionally redirects HTTP to secure HTTPS."

Want a free SSL certificate using Let's Encrypt or for deployment too?

User:

50. How to monitor metrics in Spring Boot?

ChatGPT:

Here's your **interview-ready answer** for:

? *How to monitor metrics in Spring Boot?*

"Spring Boot provides Actuator for monitoring metrics like memory, CPU, HTTP requests, DB stats, and custom health indicators via built-in endpoints."

✔ Step-by-Step Metrics Monitoring:

◆ 1. Add Spring Boot Actuator Dependency

```
xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

◆ 2. Enable Required Endpoints in application.properties

```
properties
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
management.endpoint.metrics.enabled=true
```

◆ 3. Access Built-in Metrics Endpoints

Endpoint	Description
/actuator/health	App health status
/actuator/metrics	All available metrics
/actuator/metrics/jvm.memory.used	Memory usage
/actuator/metrics/http.server.requests	HTTP request stats
/actuator/beans	List of Spring beans

◆ 4. Monitor in Real-Time (Tools)

- **Prometheus + Grafana** → Visual dashboards
- **Micrometer** → Under-the-hood metric collection abstraction used by Actuator
- **Cloud tools** → New Relic, Datadog, AWS CloudWatch integration

◆ 5. Add Custom Metrics (Optional)

```
java
@Autowired
private MeterRegistry meterRegistry;

@PostConstruct
public void customMetric() {
    meterRegistry.counter("custom.api.calls").increment();
}
```

🚀 One-liner to impress:

"Spring Boot Actuator exposes real-time app metrics via endpoints like /actuator/metrics, and integrates with tools like Prometheus and Grafana for production monitoring."

Need help integrating it with Prometheus or custom health checks too?

User:

51. What is @Transactional?

ChatGPT:

Here's your **interview-winning answer** for:

? What is @Transactional in Spring Boot?

"@Transactional is used to manage database transactions. It ensures that a group of operations either all succeed or all fail — maintaining data consistency."

✅ What It Does:

- Begins a transaction before method execution.
- Commits transaction if method runs successfully.
- Rolls back transaction if any runtime (unchecked) exception is thrown.

✅ Simple Usage:

```
java
@Service
public class AccountService {

    @Autowired private AccountRepository repo;

    @Transactional
    public void transferMoney(Long fromId, Long toId, double amount) {
        Account from = repo.findById(fromId).get();
        Account to = repo.findById(toId).get();

        from.setBalance(from.getBalance() - amount);
        to.setBalance(to.getBalance() + amount);

        repo.save(from);
        repo.save(to);
    }
}
```

If anything fails (like DB error), both updates are rolled back automatically.

✅ Rollback Rules:

- Rolls back on **RuntimeException** (unchecked)
- Won't roll back on **checked exceptions** unless specified:

```
java
@Transactional(rollbackFor = Exception.class)
```

◆ Where to Place @Transactional?

- On service methods (@Service)
- Not recommended directly on repository or controller layer

🚀 One-liner to impress:

"@Transactional ensures data integrity by committing all DB operations only if everything in the method succeeds, else it rolls back."

Want to see it in action with a custom rollback demo or nested transactions?

User:

52. What is lazy initialization in Spring Boot?

ChatGPT:

Here's your **interview-cracking, beginner-friendly answer** for:

? **What is lazy initialization in Spring Boot?**

"Lazy initialization means Spring will create the bean only when it's actually needed, not at application startup."

✅ **Default Behavior:**

- By default, Spring Boot eagerly creates all singleton beans at startup (even if they are never used).

✅ **Lazy Initialization Use Case:**

Useful when:

- You want faster startup time.
- You have optional or rarely used beans.
- You want to avoid creating heavy/unused objects unnecessarily.

✅ **How to Enable Lazy Initialization Globally:**

In `application.properties`:

```
properties
spring.main.lazy-initialization=true
```

✅ **How to Use Lazy Init on Specific Bean:**

```
java
@Component
@Lazy
public class HeavyComponent {
    public HeavyComponent() {
        System.out.println("HeavyComponent Loaded");
    }
}
```

Spring will now create `HeavyComponent` **only** when injected or accessed.

🔥 **Example:**

```
java
@Service
public class TestService {
    @Autowired
    @Lazy
    private HeavyComponent heavyComponent;
}
```

- `HeavyComponent` is not initialized until `TestService` actually uses it.

✳️ **One-liner to impress:**

"Lazy initialization delays bean creation until it's needed — improving startup time and memory usage in large Spring Boot apps."

Want to compare startup performance with and without lazy loading?

User:

53. What is Spring Boot CLI?

ChatGPT:

Here's your **interview-ready answer** for:

? **What is Spring Boot CLI?**

"Spring Boot CLI (Command Line Interface) is a tool that lets you quickly run and test Spring Boot applications using Groovy scripts without needing full Java code or IDE setup."

✓ **Key Features:**

- Run Spring Boot apps from terminal with `spring run`
- Write apps in **Groovy** (less boilerplate than Java)
- Auto-detects dependencies based on code (no need for `pom.xml`)
- Useful for **prototyping**, **learning**, or **microservices demos**

✓ **Example Groovy App (hello.groovy):**

```
groovy
@RestController
class HelloController {
    @RequestMapping("/")
    String home() {
        "Hello from Spring Boot CLI!"
    }
}
```

Run it with:

```
bash
spring run hello.groovy
```

No `main()` method, no classpath config — just works.

✓ **How to Install CLI:**

If using **SDKMAN**:

```
bash
sdk install springboot
```

Or via **curl**:

```
bash
curl -s https://get.sdkman.io | bash
```

✦ **One-liner to impress:**

"Spring Boot CLI is a lightweight way to build and run Spring apps using Groovy from the terminal — great for rapid prototyping without full setup."

Need help running CLI on Windows/macOS or integrating it into your project flow?

User:

54. What is dependency injection?

ChatGPT:

Here's your **clear and concise interview answer** for:

? What is Dependency Injection in Spring Boot?

"Dependency Injection (DI) is a design pattern where Spring automatically provides (injects) the required dependencies (objects) to a class instead of the class creating them itself."

✔ Why It's Used:

- Reduces tight coupling between classes.
- Makes code easier to test, manage, and scale.
- Promotes **Inversion of Control (IoC)** — objects don't control their own dependencies.

✔ Example:

```
java
@Component
public class Engine {
    public String start() { return "Engine started"; }
}

@Service
public class Car {
    private final Engine engine;

    // Constructor-based DI
    @Autowired
    public Car(Engine engine) {
        this.engine = engine;
    }

    public void drive() {
        System.out.println(engine.start());
    }
}
```

- Spring auto-creates the Engine bean and injects it into Car.

✔ Types of Dependency Injection in Spring:

Type	Description
Constructor	Preferred way; ensures immutability
Setter	Injects via setters
Field (not preferred)	Injects directly on fields using @Autowired

✦ One-liner to impress:

"Dependency Injection allows Spring to manage and inject class dependencies automatically — making code loosely coupled and testable."

Want a real project example using constructor vs field injection comparison?

User:

55. Constructor vs Setter injection?

ChatGPT:

Here's your **interview-cracking answer** for:

? **Constructor vs Setter Injection in Spring Boot?**

✓ **1. Constructor Injection**

- Dependencies are injected **via constructor**.
- Ensures the object is fully initialized when created.
- Promotes **immutability** and is **preferred** by Spring.

```
java
@Service
public class Car {
    private final Engine engine;

    @Autowired
    public Car(Engine engine) {
        this.engine = engine;
    }
}
```

🔥 Best for **required dependencies**.

✓ **2. Setter Injection**

- Dependencies are injected **via setter methods**.
- Allows changing dependencies after object creation.
- Good for **optional dependencies**.

```
java
@Service
public class Car {
    private Engine engine;

    @Autowired
    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}
```

🧊 Not recommended for mandatory dependencies.

✓ **Comparison Table:**

Feature	Constructor Injection	Setter Injection
Initialization	At object creation	After object is created
Immutability	Promotes immutability	Allows re-setting dependency
Recommended for	Required dependencies	Optional dependencies
Testability	Easier (using final fields)	Slightly harder
@Autowired required?	Optional in modern Spring	Required (if multiple setters)

✳ **One-liner to impress:**

"Constructor injection is preferred in Spring because it ensures required dependencies are set and makes the class immutable and easier to test."

Need pros-cons with real-world analogy or project-level best practices too?

User:

56. How to run code only once after app starts?

ChatGPT:

Here's your **interview-perfect answer** for:

? **How to run code only once after Spring Boot app starts?**

"You can use `CommandLineRunner` or `ApplicationRunner` interfaces to run code once just after the application context is loaded."

✅ **Option 1: Using `CommandLineRunner`**

```
java
@Component
public class StartupRunner implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        System.out.println("Code runs once after app starts!");
        // your custom logic here
    }
}
```

- args = command-line arguments (optional)

✅ **Option 2: Using `ApplicationRunner`**

```
java
@Component
public class AppStartupRunner implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments args) throws Exception {
        System.out.println("Code runs after app is ready.");
    }
}
```

- `ApplicationArguments` gives access to parsed args.

✅ **When to Use:**

Use Case	Preferred Interface
Need raw string args	<code>CommandLineRunner</code>
Need structured app arguments	<code>ApplicationRunner</code>

✅ **Notes:**

- These beans run **only once**, right after the Spring context is fully loaded.
- Ideal for:
 - Logging startup messages
 - Loading default data
 - Running background setup jobs

✳️ **One-liner to impress:**

"`CommandLineRunner` and `ApplicationRunner` allow us to run custom logic once the Spring Boot app has started."

Need to restrict it to specific profile (dev, prod)? Let me know!

User:

57. How to override default properties in tests?

ChatGPT:

Here's your **interview-focused, beginner-friendly answer** for:

? **How to override default properties in Spring Boot tests?**

"We can override default properties in tests using `@TestPropertySource`, `@SpringBootTest(properties=...)`, or a custom `application-test.properties` file."

✓ **1. Using `@TestPropertySource`**

```
java
@SpringBootTest
@TestPropertySource(properties = {
    "app.mode=test",
    "db.timeout=10"
})
public class MyServiceTest {
    // overridden properties apply here
}
```

✓ **2. Using `@SpringBootTest(properties = {...})` directly**

```
java
@SpringBootTest(properties = {
    "feature.enabled=false",
    "cache.ttl=300"
})
public class FeatureTest {
}
```

✓ **3. Using `application-test.properties` file**

- Create `src/test/resources/application-test.properties`

```
properties
server.port=9999
spring.datasource.url=jdbc:h2:mem:testdb
```

- Activate it using:

```
java
@SpringBootTest
@ActiveProfiles("test")
```

✓ **Which to Use When?**

Approach	When to Use
<code>@TestPropertySource</code>	Quick test-level overrides
<code>@SpringBootTest(properties=...)</code>	Override directly in test class
<code>application-test.properties</code>	Reuse across multiple tests/profiles

✦ **One-liner to impress:**

"We override Spring Boot test properties using `@TestPropertySource`, `@SpringBootTest(properties)`, or profile-specific files like `application-test.properties`."

Want me to show how to override DB URL or mock a property-based config in a test class?

User:

59. What is `@ConditionalOnMissingBean`?

ChatGPT:

Here's your **interview-smart answer** for:

? What is `@ConditionalOnMissingBean` in Spring Boot?

"`@ConditionalOnMissingBean` is used to create a bean only if it's not already defined in the Spring context."

✓ Use Case:

- Prevents duplicate bean creation.
 - Common in **auto-configuration** classes or library modules.
-

✓ Example:

```
java
@Configuration
public class MyConfig {

    @Bean
    @ConditionalOnMissingBean(MyService.class)
    public MyService myService() {
        return new MyService(); // creates only if no MyService bean exists
    }
}
```

If somewhere else (like in your app) you've already defined a `MyService` bean, this one **will not be created**.

✓ Real Scenario:

Let's say Spring Boot auto-configures a `RestTemplate`:

```
java
@Bean
@ConditionalOnMissingBean
public RestTemplate restTemplate() {
    return new RestTemplate();
}
```

If you already have your own `RestTemplate` bean, this won't override it.

🔍 Compare With:

- `@ConditionalOnBean`: Loads bean **only if another specific bean exists**.
 - `@ConditionalOnMissingBean`: Loads bean **only if that bean is missing**.
-

🌟 One-liner to impress:

"`@ConditionalOnMissingBean` lets Spring provide a default bean only if the user hasn't already defined one — useful in auto-configuration."

Need an example with a custom fallback or default implementation?

User:

62. How to externalize configuration in Spring Boot?

ChatGPT:

Here's your **interview-cracking answer** for:

? **How to externalize configuration in Spring Boot?**

"Spring Boot allows externalizing configuration using property or YAML files, environment variables, command-line args, and more — making apps flexible for different environments."

✓ **Common Ways to Externalize Config:**

Method	Example
application.properties / .yaml	server.port=8081
Environment variables	export SERVER_PORT=9090
Command-line arguments	--server.port=8085
Custom property files	@PropertySource("classpath:my-config.properties")
@Value annotation	@Value("\${my.prop}")
@ConfigurationProperties	Binds whole class to a group of props

✓ **Example 1: application.properties**

```
properties
app.name=MyApp
```

```
java
@Value("${app.name}")
private String appName;
```

✓ **Example 2: YAML file (application.yml)**

```
yaml
app:
  name: MyApp
  version: 1.0
```

✓ **Example 3: Using @ConfigurationProperties**

```
java
@ConfigurationProperties(prefix = "app")
@Component
public class AppConfig {
    private String name;
    private String version;
    // getters/setters
}
```

✓ **External File Outside JAR (for prod):**

Run with:

```
bash
java -jar myapp.jar --spring.config.location=/etc/config/app.properties
```

✦ **One-liner to impress:**

"Spring Boot supports externalized configuration via properties/YAML, env variables, and args — enabling flexible and environment-specific setups."

Want help with profile-based external configs like dev/prod separation?

User:

63. How to load a custom configuration file?

ChatGPT:

Here's your **interview-ready answer** for:

? How to load a custom configuration file in Spring Boot?

"You can load a custom configuration file in Spring Boot using `@PropertySource`, or by specifying the file location with `spring.config.location` or `spring.config.name`."

✓ Option 1: Using `@PropertySource`

Use this when loading a custom `.properties` file from the classpath.

```
java
@Configuration
@PropertySource("classpath:myconfig.properties")
public class MyCustomConfig {

    @Value("${app.title}")
    private String appTitle;
}
```

- `myconfig.properties` should be placed under `src/main/resources`.

✓ Option 2: Using `spring.config.name` or `spring.config.location`

► In `application.properties`:

```
properties
spring.config.name=myconfig
```

Looks for `myconfig.properties` or `myconfig.yml` in classpath.

► Or while running the app:

```
bash
java -jar app.jar --spring.config.location=classpath:/custom-config.properties
```

Or from filesystem:

```
bash
java -jar app.jar --spring.config.location=file:/opt/configs/app-prod.properties
```

✓ Note:

- If you're using `spring.config.location`, it completely **overrides** default `application.properties`.
- You can also use multiple files:

```
bash
--spring.config.location=file:/default.properties,file:/override.properties
```

✳ One-liner to impress:

"To load a custom config file, use `@PropertySource` in code or `spring.config.location` to specify it at runtime."

Need help binding YAML configs with `@ConfigurationProperties` from a custom file?

User:

64. How to log SQL queries in Spring Boot with JPA?

ChatGPT:

Here's your **interview-focused and easy-to-understand answer** for:

? **How to log SQL queries in Spring Boot with JPA?**

"To log SQL queries in Spring Boot JPA, enable Hibernate logging in your `application.properties` or `application.yml`."

✓ **For `application.properties`:**

```
properties
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
```

✓ **For `application.yml`:**

```
yml
spring:
  jpa:
    show-sql: true
    properties:
      hibernate:
        format_sql: true
logging:
  level:
    org.hibernate.SQL: DEBUG
    org.hibernate.type.descriptor.sql.BasicBinder: TRACE
```

🔍 **What each does:**

Property	Purpose
<code>spring.jpa.show-sql=true</code>	Prints raw SQL queries
<code>hibernate.format_sql=true</code>	Formats SQL for readability
<code>org.hibernate.SQL=DEBUG</code>	Logs SQL at DEBUG level
<code>org.hibernate.type.descriptor.sql.BasicBinder=TRACE</code>	Logs query parameters (values)

📌 **One-liner to impress:**

"We log SQL queries in Spring Boot by enabling `spring.jpa.show-sql` and setting Hibernate logging levels to **DEBUG** and **TRACE**."

Need a sample output or how to log to file instead of console?

User:

65. How to enable H2 console in Spring Boot?

ChatGPT:

Here's your **crack-the-interview** answer for:

? **How to enable H2 console in Spring Boot?**

"You can enable the H2 database console using Spring Boot properties, which lets you access an in-memory DB via browser for debugging or testing."

✔ Step-by-step (in application.properties):

```
properties
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
```

✔ For application.yml:

```
yml
spring:
  h2:
    console:
      enabled: true
      path: /h2-console
  datasource:
    url: jdbc:h2:mem:testdb
    driver-class-name: org.h2.Driver
    username: sa
    password:
```

✔ How to Access Console:

- Start your Spring Boot app.
- Visit: `http://localhost:8080/h2-console`
- Use JDBC URL: `jdbc:h2:mem:testdb`

🔒 Pro Tip (Security):

If using Spring Security, allow access to `/h2-console/**` in your security config.

```
java
http.csrf().disable()
    .headers().frameOptions().disable()
    .and()
    .authorizeRequests()
    .antMatchers("/h2-console/**").permitAll();
```

✳ One-liner to impress:

"To enable H2 console in Spring Boot, set `spring.h2.console.enabled=true` and access it via `/h2-console` in the browser."

Want to use file-based H2 DB or connect H2 from external DB tool?

User:

66. What is `@EntityScan` used for?

ChatGPT:

Here's your **interview-polished, clear answer** for:

? What is `@EntityScan` used for in Spring Boot?

"`@EntityScan` tells Spring Boot where to scan for JPA entity classes (i.e., classes annotated with `@Entity`)."

✔ Why Needed?

By default, Spring Boot only scans for entities in the **same package** or **sub-packages** of your main class.

If your entities are in a different package, you must explicitly tell Spring Boot where to look using `@EntityScan`.

✔ Syntax:

```
java
@SpringBootApplication
@EntityScan(basePackages = "com.example.entities")
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

✔ Example Scenario:

- Your main class is in: `com.app.main`
- Your entities are in: `com.app.model`

Spring Boot won't find them unless:

```
java
@EntityScan("com.app.model")
```

✦ One-liner to impress:

"@EntityScan is used to specify the packages where Spring should look for JPA entity classes if they're outside the default scan path."

Need example where `@EntityScan` and `@ComponentScan` are used together?

User:

67. What is the difference between `CrudRepository`, `JpaRepository`, and `PagingAndSortingRepository`?

ChatGPT:

Here's your **interview-strong and beginner-friendly** answer for:

? **What is the difference between `CrudRepository`, `JpaRepository`, and `PagingAndSortingRepository`?**

All are Spring Data interfaces used for database operations, but each adds extra features on top of the previous one.

✔ 1. `CrudRepository<T, ID>`

Basic CRUD operations

- Methods: `save()`, `findById()`, `findAll()`, `deleteById()`, etc.

```
java
public interface UserRepository extends CrudRepository<User, Long> {}
```

✔ 2. `PagingAndSortingRepository<T, ID>`

Extends `CrudRepository` + adds **pagination & sorting**

- Extra Methods: `findAll(Pageable pageable)`, `findAll(Sort sort)`

```
java
public interface UserRepository extends PagingAndSortingRepository<User, Long> {}
```

✔ 3. `JpaRepository<T, ID>`

Extends `PagingAndSortingRepository` + **JPA-specific features**

- Extra Methods: `flush()`, `saveAll()`, `findAllById()`, `findById()`, etc.
- Most commonly used in real projects.

```
java
public interface UserRepository extends JpaRepository<User, Long> {}
```

📄 Interface Hierarchy:

```
nginx
JpaRepository → PagingAndSortingRepository → CrudRepository
```

🌟 One-liner to impress:

"CrudRepository provides basic CRUD, PagingAndSortingRepository adds pagination, and JpaRepository gives full JPA features — it's the most powerful and commonly used."

Want me to show which one to use in a real project or pagination example?

User:

68. What is OpenFeign in Spring Boot?

ChatGPT:

Here's your **interview-crackable, no-jargon answer** for:

? *What is OpenFeign in Spring Boot?*

"OpenFeign is a declarative REST client in Spring Boot that lets you call other microservices just by writing an interface — no need to write RestTemplate code manually."

✅ Key Points:

- You define an interface and annotate it.
- Feign auto-creates the implementation at runtime.
- Cleaner and easier than RestTemplate or WebClient.

✅ Basic Example:

```
java
@FeignClient(name = "user-service")
public interface UserClient {
    @GetMapping("/users/{id}")
    User getUserById(@PathVariable("id") Long id);
}
```

Now you can **autowire and use** it:

```
java
@Autowired
private UserClient userClient;

User user = userClient.getUserById(1L);
```

✅ Enable Feign:

In your main class:

```
java
```



```
@SpringBootApplication
@EnableFeignClients
public class MyApp { ... }
```

✅ Why use OpenFeign?

Benefit	Description
Declarative style	Just write an interface
Built-in load balancing	Integrates with Eureka/Discovery
Easy to maintain	Less boilerplate than RestTemplate
Works well with Spring Cloud	Great for microservices architecture

📌 One-liner to impress:

"OpenFeign simplifies calling REST APIs by turning HTTP calls into Java interface methods — ideal for microservice communication."

Want to see how it integrates with Eureka or pass headers like Authorization (JWT)?

User:

69. What is circuit breaker in microservices?

ChatGPT:

Here's your **interview-smart, beginner-friendly answer** for:

? What is Circuit Breaker in Microservices?

"Circuit Breaker is a design pattern that prevents a microservice from repeatedly trying to call a failing service — it helps avoid cascading failures and improves system stability."

✅ Why it's needed?

- In microservices, one slow/failing service can bring others down.
- Circuit breaker detects failures and **stops calling** the failed service for a while.
- Gives the service time to recover and avoids wasting resources.

✅ How it works:

State	Behavior
Closed	Calls the remote service normally
Open	Skips the call immediately, returns fallback
Half-Open	Sends a test call to check if service is healthy again

If the test passes → closes circuit; else → keeps it open.

✅ Tools Used in Spring Boot:

- **Resilience4j** (modern, lightweight)
- **Hystrix** (Netflix, now in maintenance mode)

✅ Resilience4j Example:

```
java
@CircuitBreaker(name = "userService", fallbackMethod = "fallbackUser")
public User getUser(Long id) {
    return restTemplate.getForObject("http://user-service/users/" + id, User.class);
}
```

```
public User fallbackUser(Long id, Throwable t) {  
    return new User(id, "Default User", "N/A");  
}
```

✦ One-liner to impress:

"Circuit Breaker avoids system crashes by breaking the call chain to a failing service and returning a fallback response instead."

Want me to give complete config steps using Resilience4j in Spring Boot?

User:

70. What is Spring Cloud Config?

ChatGPT:

Here's your **interview-focused and beginner-friendly answer** for:

? What is Spring Cloud Config?

"Spring Cloud Config is a centralized configuration management tool for microservices — it lets multiple services share externalized properties from one place."

✓ Why Use It?

- In microservices, each service has its own config (ports, DB URLs, API keys, etc.).
- Managing configs separately becomes hard.
- Spring Cloud Config lets you store all configs **in a Git repo or file server**, and services fetch them dynamically.

✓ Key Features:

- Centralized config server
- Environment-specific configs (e.g. application-dev.yml)
- Git-based or file-based backend
- Runtime refresh (with Spring Cloud Bus + Actuator)

✓ How It Works:

1. **Config Server** — A Spring Boot app that exposes configs via REST (acts as the centralized server).
2. **Config Client** — Your microservice that connects to the config server to fetch its own config.

✓ Sample Setup:

Config Server:

```
java  
@SpringBootApplication  
@EnableConfigServer  
public class ConfigServerApp {}
```

application.yml:

```
yaml  
server:  
  port: 8888  
spring:  
  cloud:  
    config:  
      server:
```

```
git:
  uri: https://github.com/my-org/config-repo
```

Config Client (your microservice):

bootstrap.yml:

```
yaml
spring:
  application:
    name: orderservice
  cloud:
    config:
      uri: http://localhost:8888
```

🚀 One-liner to impress:

"Spring Cloud Config provides a central config server so all microservices can pull their settings from one place — reducing duplication and improving manageability."

Want steps for environment-specific Git folders or auto-refreshing configs without restarting?

User:

71. How to make Spring Boot app stateless?

ChatGPT:

Here's your **interview-smart and easy-to-understand** answer for:

? How to make a Spring Boot app stateless?

"To make a Spring Boot app stateless, avoid using HTTP sessions and store all authentication state on the client side (like using JWT tokens)."

✅ What is Stateless?

- **Stateless** means: **Server does not store any user/session data between requests.**
- Each request should carry all the required information (like token).

✅ Key Steps to Make Spring Boot App Stateless:

1. Disable Session Creation

In your SecurityConfig:

```
java
http
.sessionManagement()
.sessionCreationPolicy(SessionCreationPolicy.STATELESS);
```

2. Use JWT for Authentication

- Generate token on login
- Send token in Authorization header (e.g., Bearer ey...)
- Validate token on each request (no session used)

3. Do Not Use @SessionAttributes / HttpSession

- Don't store user data in HttpSession, cookies, or server memory.

✅ Why Go Stateless?

Benefit	Description
Scalable	No session memory = easy to scale horizontally
Secure	Token-based auth (e.g., JWT) is safer

Benefit	Description
Microservices friendly	Ideal for distributed/cloud environments

🔴 One-liner to impress:

"Stateless Spring Boot means disabling sessions and handling auth using tokens like JWT, where each request is independent and self-contained."

Need example of JWT filter or stateless login flow?

User:

72. How to implement pagination with Spring Data JPA?

ChatGPT:

Here's your **interview-ready, practical answer** for:

🔴 *How to implement pagination with Spring Data JPA?*

"Spring Data JPA provides built-in support for pagination using the Pageable interface — you just pass it to your repository method and it returns a Page<T>."

✅ Step-by-step Implementation:

1. Repository Method

```
java
public interface UserRepository extends JpaRepository<User, Long> {
    Page<User> findAll(Pageable pageable);
}
```

2. Service/Controller Usage

```
java
@GetMapping("/users")
public Page<User> getUsers(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "5") int size
) {
    Pageable pageable = PageRequest.of(page, size);
    return userRepository.findAll(pageable);
}
```

✅ Sample Response (automatically handled):

Spring Data JPA returns:

```
json
{
  "content": [ { ...users... } ],
  "totalPages": 10,
  "totalElements": 50,
  "size": 5,
  "number": 0
}
```

✅ Sorting Support:

You can also sort with pagination:

```
java
```

```
Pageable pageable = PageRequest.of(page, size, Sort.by("name").ascending());
```

✦ One-liner to impress:

"Pagination in Spring Data JPA is done using Pageable and Page<T>, allowing efficient loading of large data sets page-by-page."

Need example with custom query or dynamic filters + pagination?

User:

73. How to log request and response in Spring Boot REST?

ChatGPT:

Here's your **interview-friendly and practical answer** for:

? *How to log request and response in Spring Boot REST?*

"You can log requests and responses in Spring Boot using a Filter, Interceptor, or with tools like Spring Boot Actuator or Logbook."

✓ Common Approach: Use a Custom OncePerRequestFilter

```
java
@Component
public class LoggingFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        // Log Request
        String requestBody = request.getReader().lines().collect(Collectors.joining(System.lineSeparator()));
        System.out.println("REQUEST: " + request.getMethod() + " " + request.getRequestURI());
        System.out.println("Body: " + requestBody);

        // Proceed
        filterChain.doFilter(request, response);

        // Response logging can be done with wrappers (trickier)
    }
}
```

📝 Note: For **response logging**, you need to wrap the HttpServletResponse using ContentCachingResponseWrapper.

✓ Alternative: Use Spring HandlerInterceptor

```
java
@Component
public class LoggingInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) {
        System.out.println("REQUEST: " + request.getMethod() + " " + request.getRequestURI());
        return true;
    }
}
```

Register it in WebMvcConfigurer:

```
java
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Autowired LoggingInterceptor interceptor;

    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(interceptor);
    }
}
```

```
}  
}
```

✅ Third-Party Option: Use Logbook (easy plug-in)

```
xml  
<!-- pom.xml -->  
<dependency>  
  <groupId>org.zalando</groupId>  
  <artifactId>logbook-spring-boot-starter</artifactId>  
  <version>3.0.0</version>  
</dependency>
```

Automatically logs request & response beautifully.

🚀 One-liner to impress:

"Request and response logging in Spring Boot can be implemented using Filter, Interceptor, or libraries like Logbook for clean and customizable logs."

Want me to show a clean reusable Filter class that logs both request and response safely for prod use?

User:

74. Difference between @RequestParam, @PathVariable, @RequestBody?

ChatGPT:

Here's your **interview-crack style answer** with practical comparison for:

? **Difference between @RequestParam, @PathVariable, @RequestBody in Spring Boot?**

Annotation	Used For	Example URL or Usage
@RequestParam	To extract query parameters	/users?id=10
@PathVariable	To extract values from URI path	/users/10
@RequestBody	To bind request JSON/XML body to object	POST /users with JSON body

✅ Examples:

1. @RequestParam

```
java  
@GetMapping("/users")  
public String getUserById(@RequestParam int id) {  
    return "User ID: " + id;  
}  
// URL: /users?id=10
```

2. @PathVariable

```
java  
@GetMapping("/users/{id}")  
public String getUser(@PathVariable int id) {  
    return "User ID: " + id;  
}  
// URL: /users/10
```

3. @RequestBody

```
java
```

```
@PostMapping("/users")
public String createUser(@RequestBody User user) {
    return "Saved " + user.getName();
}
// Body: { "name": "John", "age": 25 }
```

✦ One-liner to impress:

"@RequestParam is for query params, @PathVariable for URL path values, and @RequestBody for reading JSON/XML payloads."

Need difference between GET vs POST vs PUT with these annotations in real project flow?

User:

75. What is spring-boot-devtools used for?

ChatGPT:

Here's your **interview-perfect and simple answer** for:

? What is spring-boot-devtools used for?

"spring-boot-devtools is a developer tool that enables automatic restart, live reload, and improved development experience in Spring Boot."

✓ Key Features:

Feature	What it does
Auto-restart	Restarts the app on code change (without full rebuild)
LiveReload	Automatically refreshes browser on front-end changes
Fast reload	Only reloads changed classes, not the whole app
Disable caching	Disables template, static file caching for faster dev
Remote dev tools	Can connect to remote app for hot reload (advanced usage)

✓ How to Use:

```
xml
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
</dependency>
```

📖 It only activates in the **development** environment (not packaged in production JAR).

✦ One-liner to impress:

"spring-boot-devtools speeds up development by auto-restarting your app and disabling caches — making code changes instantly visible."

Want tips on fixing DevTools not reloading issue in IntelliJ or VS Code?

User:

76. What is spring.main.allow-bean-definition- overriding=true?

ChatGPT:

Here's your **interview-ready, concise answer** for:

? What is `spring.main.allow-bean-definition-overriding=true`?

"`spring.main.allow-bean-definition-overriding=true` allows Spring Boot to override existing bean definitions with new ones having the same name."

✓ Explanation:

- By default, Spring **does NOT allow** two beans with the same name — it throws an error.
- Setting this property to true tells Spring to **replace the existing bean** with the new definition instead of failing.
- Useful when you want to **customize or replace beans** provided by Spring Boot or third-party libraries.

✓ When to use?

- When you have multiple configuration classes defining beans with the same name.
- To override auto-configured beans without changing original code.
- In testing scenarios to mock or replace beans.

✦ One-liner to impress:

"This property enables bean overriding to avoid startup errors when multiple beans with the same name exist, helping customize or replace default beans."

Need example of bean overriding in code?

User:

77. What is caching in Spring Boot?

ChatGPT:

Here's your **interview-ready, simple answer** for:

? What is *caching in Spring Boot*?

"Caching in Spring Boot temporarily stores frequently accessed data in memory to improve app performance and reduce repeated expensive operations."

✓ Key Points:

- **Purpose:** Avoid repeated calls to database or slow services by storing data in cache.
- **Spring Boot Support:** Uses `@EnableCaching` and cache annotations like `@Cacheable`, `@CachePut`, `@CacheEvict`.
- **Cache Providers:** Supports EhCache, Redis, Hazelcast, etc.

✓ Example:

```
java
@Cacheable("users")
public User getUserById(Long id) {
    // Expensive DB call here
    return userRepository.findById(id).orElse(null);
}
```

- First call: fetches from DB and caches result.
- Next calls: returns cached data directly.

✦ One-liner to impress:

"Caching in Spring Boot speeds up apps by storing and reusing frequently accessed data, reducing load on DB or services."

Want me to explain cache eviction and refresh strategies?

User:

78. How to enable async execution? what is async

ChatGPT:

Here's your **interview-ready, clear answer** for:

? What is async and how to enable async execution in Spring Boot?

What is Async?

Async (Asynchronous) execution means running tasks in the background without blocking the main thread, so the app can handle other requests simultaneously.

How to Enable Async in Spring Boot?

1. Add `@EnableAsync` on any configuration class:

```
java
@Configuration
@EnableAsync
public class AsyncConfig {
}
```

2. Mark methods you want to run asynchronously with `@Async`:

```
java
@Async
public void sendEmail(String user) {
    // runs in separate thread
}
```

3. Call async method — it runs in background, main thread is free immediately.

Why use async?

- Improves app responsiveness.
- Useful for tasks like sending emails, processing files, or calling external APIs.

✦ One-liner to impress:

"Async in Spring Boot runs methods in separate threads using `@Async` and `@EnableAsync` to improve performance by not blocking the main flow."

Want me to share how to customize thread pool for async tasks?

User:

80. What is actuator endpoint `/env` used for? in 1 line ans only for this

ChatGPT:

`/env` actuator endpoint exposes current environment properties and configuration details of the Spring Boot application.

User:

81. What is the difference between Mono and Flux in WebFlux?

ChatGPT:

Here's your **interview-ready quick answer** for:

? *Difference between Mono and Flux in Spring WebFlux?*

Aspect	Mono	Flux
Represents	0 or 1 item	0 to N items
Use case	Single async value or empty	Multiple async values or stream
Example	<code>Mono<String></code>	<code>Flux<String></code>

🚀 **One-liner to impress:**

"Mono handles single or no result asynchronously, while Flux handles multiple asynchronous results as a reactive stream."

Want me to explain how to use them in a REST API?

User:

82. How to validate request data in Spring Boot?

ChatGPT:

Here's your **interview-ready concise answer** for:

? *How to validate request data in Spring Boot?*

How to Validate?

- Use **Bean Validation API** with annotations like `@NotNull`, `@Size`, `@Email` on your DTO/model fields.
- Annotate controller method parameter with `@Valid` to trigger validation.

Example:

```
java
public class UserDTO {
    @NotNull
    @Size(min = 2, max = 30)
    private String name;

    @Email
    private String email;
}

@PostMapping("/users")
public ResponseEntity<String> createUser(@Valid @RequestBody UserDTO user) {
    // If validation fails, Spring automatically returns 400 Bad Request
    return ResponseEntity.ok("User created");
}
```

Bonus:

- Use `@Validated` on class for group validation.
- Handle errors with `@ExceptionHandler` or `@ControllerAdvice` for custom messages.

🚀 **One-liner to impress:**

"Spring Boot validates request data using Bean Validation annotations and `@Valid` to ensure incoming data meets constraints before processing."

Want sample for custom validator?

User:

83. How to gracefully shut down Spring Boot app? also means

ChatGPT:

Here's your **interview-ready, clear answer** for:

? How to gracefully shut down a Spring Boot app? Also, what does it mean?

What does graceful shutdown mean?

"Graceful shutdown means stopping the app in a controlled way — finishing ongoing requests/tasks before closing, to avoid data loss or corruption."

How to do it in Spring Boot?

1. Spring Boot 2.x+ supports graceful shutdown out of the box.
2. Enable it in application.properties:

```
properties
server.shutdown=graceful
```

3. When app receives shutdown signal (e.g., Ctrl+C), it:

- Stops accepting new requests.
- Waits for active requests to finish within a timeout.
- Then shuts down the app cleanly.

Bonus:

- Customize timeout with `spring.lifecycle.timeout-per-shutdown-phase`.
- For beans with cleanup, implement `DisposableBean` or use `@PreDestroy`.

🔥 One-liner to impress:

"Graceful shutdown lets Spring Boot finish active tasks before stopping, preventing abrupt termination and data loss."

Want me to explain programmatic ways to trigger shutdown?

User:

84. What is reactive programming?

ChatGPT:

Here's your **interview-ready, simple answer** for:

? What is reactive programming?

"Reactive programming is a programming style focused on asynchronous data streams and the propagation of change, allowing apps to react to new data or events efficiently."

Key Points:

- Deals with **data streams** and **events** (e.g., user clicks, server responses).
- Uses **non-blocking, event-driven** approach.
- Enables building **scalable and responsive** apps.

- In Java, implemented with **Project Reactor** (Mono, Flux).

✦ One-liner to impress:

"Reactive programming lets applications handle asynchronous data streams efficiently by reacting to changes as they happen."

Want examples of reactive vs traditional code?

User:

85. How to expose only selected actuator endpoints?

ChatGPT:

Here's your **interview-ready concise answer** for:

❓ *How to expose only selected actuator endpoints in Spring Boot?*

How to do it?

- By default, only a few endpoints are exposed.
- To expose specific endpoints, configure in `application.properties` or `application.yml`:

```
properties
management.endpoints.web.exposure.include=health,info,metrics
```

- To hide endpoints, use:

```
properties
management.endpoints.web.exposure.exclude=env,beans
```

✦ One-liner to impress:

"Use `management.endpoints.web.exposure.include` property to expose only selected actuator endpoints and keep others hidden."

Want help with securing these endpoints too?

User:

86. How to auto-generate schema with JPA?

ChatGPT:

Here's your **interview-ready concise answer** for:

❓ *How to auto-generate schema with JPA in Spring Boot?*

How to do it?

- Configure Hibernate's `ddl-auto` property in `application.properties`:

```
properties
spring.jpa.hibernate.ddl-auto=update
```

- Common options:
 - `create` — creates schema at startup, drops at shutdown
 - `update` — updates schema without dropping data
 - `validate` — validates schema matches entities, no changes

- none — no action

- Spring Boot uses this to auto-create/update tables based on JPA entities.

✦ One-liner to impress:

"Spring Boot auto-generates database schema from JPA entities using `spring.jpa.hibernate.ddl-auto` property with options like `update` or `create`."

Want me to explain pros and cons of each option?

User:

87. How to ignore null fields in JSON response?

ChatGPT:

Here's your **interview-ready concise answer** for:

? *How to ignore null fields in JSON response in Spring Boot?*

How to do it?

- Configure Jackson to **ignore null values** globally by adding this in `application.properties`:

```
properties
spring.jackson.default-property-inclusion=non_null
```

- Or, use `@JsonInclude` annotation on class or fields:

```
java
@JsonInclude(JsonInclude.Include.NON_NULL)
public class User {
    private String name;
    private String email;
}
```

✦ One-liner to impress:

"Spring Boot uses Jackson's `@JsonInclude(Include.NON_NULL)` or config to exclude null fields from JSON responses, making output cleaner."

Want example for conditional field inclusion?

User:

88. What is Flyway and Liquibase?

ChatGPT:

Here's your **interview-ready concise answer** for:

? *What are Flyway and Liquibase?*

What are they?

- **Flyway** and **Liquibase** are popular database migration tools.
- They help manage **version-controlled, incremental changes** (migrations) to the database schema.
- Ensure all environments stay **in sync** and schema changes are applied safely.

Key differences:

Feature	Flyway	Liquibase
Migration approach	SQL scripts or Java-based	XML, YAML, JSON, SQL files
Flexibility	Simpler, mostly SQL scripts	More powerful, supports rollback, complex changes
Popularity	Widely used in simpler projects	Used in complex enterprise apps

🚀 One-liner to impress:

"Flyway and Liquibase automate and version-control database schema changes, ensuring safe, consistent migrations across environments."

Want me to share Spring Boot integration example?

User:

90. How to encrypt properties in Spring Boot?

ChatGPT:

Here's your **interview-ready concise answer** for:

🔍 *How to encrypt properties in Spring Boot?*

How to do it?

- Use **Spring Cloud Config with encryption support** or third-party libs like **Jasypt**.
- With **Jasypt**, encrypt property values and decrypt at runtime.

Example with Jasypt:

1. Add dependency:

```
xml
<dependency>
  <groupId>com.github.ulisesbocchio</groupId>
  <artifactId>jasypt-spring-boot-starter</artifactId>
  <version>3.0.4</version>
</dependency>
```

2. Encrypt property value and put in application.properties:

```
properties
spring.datasource.password=ENC(encryptedPasswordHere)
```

3. Provide encryption password as environment variable:

```
bash
export JASYPT_ENCRYPTOR_PASSWORD=yourSecretKey
```

🚀 One-liner to impress:

"Spring Boot supports property encryption using Jasypt or Spring Cloud Config, securing sensitive data like passwords at runtime."

Want me to explain steps to generate encrypted values?

User:

91. How to reload properties at runtime?

ChatGPT:

Here's your **interview-ready concise answer** for:

? *How to reload properties at runtime in Spring Boot?*

How to do it?

- Use **Spring Cloud Config** with `@RefreshScope` on beans to reload config without restarting app.
 - Trigger reload by calling `/actuator/refresh` endpoint (requires actuator and spring-cloud dependencies).
 - Alternatively, use third-party libs like **Spring Boot DevTools** for local reloads.
-

Example:

```
java
@RefreshScope
@Component
public class MyBean {
    @Value("${my.property}")
    private String myProperty;
}
```

✦ One-liner to impress:

"Spring Boot reloads properties at runtime using Spring Cloud Config with `@RefreshScope` and `/actuator/refresh` endpoint without app restart."

Want me to share how to enable `/actuator/refresh`?

User:

92. What is `@Profile` annotation?

ChatGPT:

Here's your **interview-ready concise answer** for:

? *What is `@Profile` annotation in Spring Boot?*

What is it?

- `@Profile` is used to **activate beans conditionally** based on the current active environment/profile (e.g., dev, test, prod).
 - Helps load **environment-specific beans/configurations**.
-

How it works?

- Annotate a class or method with `@Profile("dev")` — this bean loads only when dev profile is active.
 - Multiple profiles can be specified.
-

Example:

```
java
@Profile("dev")
@Service
public class DevService {
    // Bean active only in 'dev' profile
}
```

✦ One-liner to impress:

"`@Profile` lets Spring Boot load beans conditionally based on active environment profiles, enabling environment-specific configurations."

Want me to explain how to set active profiles?

User:

93. How to enable hot reloading in IntelliJ or Eclipse?

ChatGPT:

Here's your **interview-ready concise answer** for:

? How to enable hot reloading in IntelliJ or Eclipse for Spring Boot?

How to do it?

- Use **Spring Boot DevTools** dependency for automatic restart & live reload.
- In **IntelliJ**:
 - Enable **Build project automatically** (Ctrl+Shift+A → search "Build project automatically").
 - Enable **Allow auto-make to start even if the app is running** in **Compiler** settings.
- In **Eclipse**:
 - Enable **Build Automatically** from menu.
- DevTools detects code changes and restarts app quickly without full rebuild.

Add DevTools dependency:

```
xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <optional>true</optional>
</dependency>
```

✦ One-liner to impress:

"Spring Boot DevTools with IDE auto-build enables hot reloading, speeding up development by restarting app on code changes automatically."

Want setup steps for VS Code or other IDE?

User:

94. What is @RequestScope, @SessionScope, @ApplicationScope?

ChatGPT:

Here's your **interview-ready concise answer** for:

? What are @RequestScope, @SessionScope, and @ApplicationScope in Spring Boot?

What are they?

These annotations define **lifecycle scopes** of Spring beans in a web application:

Scope	Description	Bean Lifetime
@RequestScope	Bean created once per HTTP request	Lives only during a single HTTP request
@SessionScope	Bean created per user session	Lives as long as user session is active
@ApplicationScope	Bean shared across the entire application (all users)	Lives as long as the app runs

Usage:

```
java
@RequestScope
@Component
public class RequestBean { ... }

@SessionScope
@Component
public class SessionBean { ... }

@ApplicationScope
@Component
public class AppBean { ... }
```

✦ One-liner to impress:

"These scopes control bean lifecycle per request, user session, or entire app, enabling proper resource management in web apps."

Want me to add example use cases?

User:

95. What is role of @ComponentScan?

ChatGPT:

Here's your **interview-ready concise answer** for:

🔍 *What is the role of @ComponentScan in Spring Boot?*

What it does?

- @ComponentScan tells Spring **where to look for components** (like @Component, @Service, @Repository, @Controller) to auto-detect and register as beans.
- By default, Spring Boot scans the package where the main application class is located and its sub-packages.

Why important?

- Without it, Spring won't know which classes to manage as beans.
- Helps modularize your app by scanning specific packages.

Example:

```
java
@ComponentScan(basePackages = "com.example.service")
public class AppConfig { }
```

✦ One-liner to impress:

"@ComponentScan enables Spring to automatically discover and register beans in specified packages for dependency injection."

Want me to explain default behavior in Spring Boot?

User:

96. What is Spring Boot Admin?

ChatGPT:

Here's your **interview-ready concise answer** for:

? **What is Spring Boot Admin?**

What is it?

- Spring Boot Admin is a community project that provides a **web UI to monitor and manage Spring Boot applications**.
 - It aggregates health, metrics, logs, and other actuator endpoints of multiple apps in one dashboard.
-

Key features:

- Centralized monitoring of multiple Spring Boot apps.
 - Shows health, metrics, environment, and thread dumps.
 - Supports notifications on app status changes.
 - Easy integration using actuator endpoints.
-

✦ **One-liner to impress:**

"Spring Boot Admin provides a centralized dashboard to monitor and manage multiple Spring Boot apps via actuator endpoints."

Want me to share basic setup steps?

User:

97. What is the difference between microservices and monolith in Spring Boot?

ChatGPT:

Here's your **interview-ready concise answer** for:

? **Difference between Microservices and Monolith in Spring Boot?**

Aspect	Monolith	Microservices
Architecture	Single, unified application	Multiple small, independent services
Deployment	Deployed as one unit	Each service deployed separately
Scalability	Scale entire app	Scale individual services as needed
Development	Simple to develop initially	More complex, requires service coordination
Technology	Usually one tech stack	Can use different tech per service
Fault Isolation	Failure affects whole app	Failures isolated to services

✦ **One-liner to impress:**

"Monolith is a single unified app, while microservices break functionality into independent deployable services for better scalability and flexibility."

Want me to explain pros and cons?

User:

98. How to run Spring Boot app inside Docker?

ChatGPT:

Here's your **interview-ready concise answer** for:

? **How to run Spring Boot app inside Docker?**

Steps:

1. **Create a Dockerfile** in your project root:

```
dockerfile
FROM openjdk:17-jdk-alpine
VOLUME /tmp
COPY target/your-app.jar app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

2. **Build your Spring Boot jar:**

```
bash
mvn clean package
```

3. **Build Docker image:**

```
bash
docker build -t your-app-name .
```

4. **Run Docker container:**

```
bash
docker run -p 8080:8080 your-app-name
```

🚀 One-liner to impress:

"Dockerizing Spring Boot app involves creating a Dockerfile, building the jar, creating an image, and running a container exposing required ports."

Want a Docker Compose example too?

User:

99. What are disadvantages of Spring Boot?

ChatGPT:

Here's your **interview-ready concise answer** for:

? **What are the disadvantages of Spring Boot?**

Disadvantages:

- **Large executable size** — fat JARs can be heavy, increasing deployment size.
- **Hidden complexity** — auto-configuration can sometimes hide what's going on, making debugging harder.
- **Less control** — automatic setup may not suit very fine-tuned or custom configurations.
- **Startup time** — can be slower than lightweight frameworks due to many dependencies.
- **Learning curve** — beginners may struggle with all conventions and magic happening behind the scenes.

🚀 One-liner to impress:

"Spring Boot's auto-configuration and fat jars add convenience but can increase size, reduce transparency, and sometimes make debugging harder."

Want me to suggest ways to mitigate these disadvantages?

User:

100. What are new features in Spring Boot 3.x?

ChatGPT:

Here's your **interview-ready concise answer** for:

? What are new features in Spring Boot 3.x?

Key new features in Spring Boot 3.x:

- **Java 17+ baseline** — requires Java 17 or higher, leveraging new language features.
 - **Spring Framework 6** — upgraded core with native support for Jakarta EE 9+ namespaces.
 - **Native AOT (Ahead-of-Time) compilation support** — better native image generation with GraalVM for faster startup.
 - **Improved observability** — enhanced Micrometer metrics and tracing integration.
 - **Updated dependencies** — latest versions of Hibernate, Spring Data, and others.
 - **Removed deprecated features** — cleaner, modern APIs.
 - **Kotlin coroutines support** — improved for reactive programming.
 - **Improved support for GraalVM native images** — for cloud-native apps.
-

📌 One-liner to impress:

"Spring Boot 3.x supports Java 17+, Spring Framework 6, native AOT compilation, improved observability, and better cloud-native features."

Want me to share migration tips from 2.x to 3.x?
